

# Claude Code Deep Dive Workshop

## Chapter 4 - Settings

settings.json, Hooks, MCP 통합 심화

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

[whchoi@amazon.com](mailto:whchoi@amazon.com)



# Agenda

## Chapter 4 / Settings

- 01 settings.json 구조  
3단 계층, 모든 옵션, 병합 규칙
- 02 Permissions 심화  
allow/ask/deny, 그룹과 명령 패턴
- 03 Hooks 4가지 시점  
PreToolUse, PostToolUse, Session, Stop
- 04 Hooks 실전 패턴  
5가지 자동화 시나리오 구현
- 05 MCP 기본과 공식 서버  
filesystem, github, slack, jira
- 06 MCP 사내 서버 + Commands  
TypeScript SDK, 슬래시 명령
- 07 Recap & 4 Labs  
핵심 정리와 4가지 실습

## CHAPTER 04

# Settings

settings.json, Hooks, MCP 심화



# Learning Objectives

## Power User가 챗터를 마치고 할 수 있는 것

### OBJ 1

#### settings.json 완전 이해

permissions, model, hooks, mcp 등 모든 옵션을 명확히 설정.

### OBJ 2

#### Hooks 자동화

PreToolUse, PostToolUse, SessionStart, Stop 4가지 시점을 활용한 자동화를 구현.

### OBJ 3

#### MCP 통합

공식 서버를 추가하고 사내 MCP 서버를 직접 작성해 Claude Code와 연동.

### OBJ 4

#### Custom Commands

슬래시 명령을 작성해 팀의 반복 작업을 한 명령으로 자동화.

### OUTCOME

챗터를 마치면 본인 워크플로에 맞게 Claude Code를 깊이 커스터마이징하고, 팀 표준 자동화 자산을 구축할 수 있게 됩니다.

4

학습 목표

9 Parts

구성

4 Labs

실습 랩

Power user

대상 수준

# Power User 사고방식

## 기본 사용에서 한 단계 더

### STAGE 1

#### 기본 사용

claude 명령으로 대화하고  
코드를 작성.  
기본 도구만 활용.

### STAGE 2

#### 설정 최적화

settings.json으로 자동 허  
용 도구를 늘리고 자주 쓰는  
모델을 고정.

### STAGE 3

#### 자동화 구축

Hooks와 슬래시 명령으로  
반복 작업을 자동화.

### STAGE 4

#### 도구 통합

MCP 서버를 작성해 사내 시  
스템과 Claude Code를 직  
접 연결.

### THE GOAL

이번 챕터는 Stage 1에서 Stage 4까지의 단계적 진입을 다룹니다. 본인 워크플로에 가장 잘 맞는 수준까지 끌어올리는 것이 목표입니다.

Basic  
Stage 1

Tuned  
Stage 2

Automated  
Stage 3

Integrated  
Stage 4

## CHAPTER 04 – PART 01

# Settings – settings.json 구조

settings.json, Hooks, MCP 심화



# Part 1 / settings.json 구조

## 3단 계층과 모든 옵션 이해

### PART OBJECTIVE

settings.json의 3단 계층(managed/project/user), 모든 옵션 카테고리, 병합 규칙, 우선순위 결정을 단계별로 학습.

#### TOPIC 1

##### 3단 계층

managed-settings,  
.claude/settings.json,  
~/.claude/settings.json

#### TOPIC 2

##### 옵션 카테고리

permissions, model,  
hooks, mcp, telemetry,  
theme 6가지

#### TOPIC 3

##### 병합 규칙

deny는 항상 승리,  
allow/ask는 상위 우선, 배열  
추가 규칙

#### TOPIC 4

##### 실전 예시

개인 사용자, 팀 프로젝트,  
조직 표준 3가지 완성 예시

15 Slides

Part 1 분량

~30 min

예상 강의 시간

3 levels

계층 구조

6 categories

옵션 영역

# 3단 계층 구조

## 파일 위치와 우선순위

Hierarchy

# 우선순위 (높음 → 낮음)

# Tier 1: managed-settings.json (조직 강제, 최우선)

macOS: /Library/Application Support/Claude/managed-settings.json

Linux: /etc/claude/managed-settings.json

Windows: C:\ProgramData\Claude\managed-settings.json

권한: root/Administrator 필요

용도: 회사 정책 강제

# Tier 2: 프로젝트 settings (.claude/settings.json)

위치: 프로젝트 루트의 .claude/settings.json

권한: 프로젝트 팀이 Git으로 관리

용도: 프로젝트 특수 설정 (팀 공유)

# Tier 3: 사용자 settings (~/.claude/settings.json)

위치: 사용자 홈 디렉토리

권한: 본인만 수정 가능

용도: 개인 선호 (테마, 단축키, 개인 MCP)

# 추가: settings.local.json (Git 미커밋 권장)

프로젝트 .claude/settings.local.json

용도: 개인의 프로젝트별 오버라이드

managed

조직 (root)

project

팀 공유

user

개인 전역

local

개인 프로젝트



# settings.json 전체 옵션

## 6대 카테고리

All options

```
{
  "permissions": {
    "allow": [...], "ask": [...], "deny": [...]
  },

  "model": "claude-sonnet-4-5",
  "modelAlias": {
    "fast": "claude-haiku-4-5",
    "smart": "claude-opus-4-5"
  },

  "hooks": {
    "PreToolUse": [...],
    "PostToolUse": [...],
    "SessionStart": [...],
    "Stop": [...]
  },

  "mcpServers": {
    "filesystem": { ... },
    "github": { ... },
    "internal-jira": { ... }
  },

  "telemetry": {
    "enabled": true,
    "endpoint": "https://otel.company.com"
  },

  "theme": "dark",
  "editor": { "tabWidth": 2, "insertFinalNewline": true },
  "outputStyle": "concise"
}
```

permissions

권한

hooks

자동화

mcp

도구 통합

model

모델 선택

# permissions 옵션

## 도구 권한 통제의 핵심

```
{
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob",
      "Bash(npm test:*)",
      "Bash(npm run lint)",
      "Bash(git diff:*)",
      "Read(src/**)",
      "Edit(docs/**)"
    ],
    "ask": [
      "Edit(src/**)",
      "Write(**)",
      "Bash(git push:*)",
      "Bash(git commit:*)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Bash(curl:*)",
      "Bash(* > /etc/*)",
      "Read(**/.env*)",
      "Read(**/secrets/**)"
    ],
    "additionalDirectories": [
      "/Users/me/shared-libs"
    ]
  }
}
```

permissions

**allow**

자동 허용

**ask**

사용자 확인

**deny**

강제 차단

**additionalDirs**

외부 디렉토리

# model 옵션

## 모델 선택과 별명

```
{
  "model": "claude-sonnet-4-5",

  "modelAlias": {
    "fast": "claude-haiku-4-5",
    "smart": "claude-opus-4-5",
    "default": "claude-sonnet-4-5"
  }
}
```

### # 사용 방법

# 1. 기본 모델: model 필드 값 사용

# 2. 명시 호출: > /model smart → Opus 사용

# 3. Subagent에서 사용:

# .claude/agents/my-agent.md frontmatter에

# model: smart

### # 비용 통제 패턴

# managed-settings에서 model 강제:

```
{
  "model": "claude-sonnet-4-5",
  "modelAlias": {
    "smart": "claude-sonnet-4-5" # smart도 sonnet으로
  }
}
```

# 사용자가 smart를 호출해도 Sonnet 사용

### # 모델 차단

# managed-settings deny:

```
{
  "deniedModels": ["claude-opus-4-5"]
}
```

model

model

기본 모델

alias

별명

/model

런타임 전환

deny

모델 차단

# hooks 옵션 개요

## 4가지 시점의 자동화

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "/opt/check.sh"
        }]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write",
        "hooks": [{
          "type": "command",
          "command": "prettier --write $(jq -r '.tool_input.file_path')"
        }]
      }
    ]
  },
}
```

```
"SessionStart": [
  {
    "hooks": [{
      "type": "command",
      "command": "echo 'Session started'"
    }]
  },
],
"Stop": [
  { "hooks": [{ "type": "command", "command": "/opt/cleanup.sh" }] }
]
}
```

hooks

**PreToolUse**

도구 호출 전

**PostToolUse**

도구 호출 후

**SessionStart**

세션 시작

**Stop**

응답 종료

# mcpServers 옵션

## 3가지 통신 방식

mcpServers

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/path"],
      "env": {}
    },

    "github": {
      "type": "http",
      "url": "https://mcp-github.example.com",
      "headers": {
        "Authorization": "Bearer ${GITHUB_TOKEN}"
      }
    },

    "internal-jira": {
      "type": "sse",
      "url": "https://mcp-jira.company.com/sse",
      "headers": {
        "X-Auth": "${JIRA_TOKEN}"
      }
    },

    "custom-stdio": {
      "command": "/opt/my-mcp/server",
      "args": ["--config", "/etc/mcp.json"],
      "env": {
        "LOG_LEVEL": "info"
      }
    }
  }
}
```

stdio

로컬 프로세스

http

HTTP API

sse

Server-Sent

env vars

환경변수 주입

# telemetry, theme, editor

## 운영과 사용자 경험 옵션

misc

```
{
  "telemetry": {
    "enabled": true,
    "endpoint": "https://otel.company.com:4318",
    "headers": { "Authorization": "Bearer ${OTEL_TOKEN}" },
    "exporters": ["traces", "metrics", "logs"],
    "sampling": { "traces": 1.0, "metrics": "always" },
    "attributes": {
      "service.name": "claude-code",
      "deployment.environment": "production"
    }
  },

  "theme": "dark", // "dark" | "light" | "system"

  "editor": {
    "tabWidth": 2,
    "insertFinalNewline": true,
    "trimTrailingWhitespace": true,
    "preferredQuotes": "single"
  },

  "outputStyle": "concise", // "concise" | "verbose" | "casual"

  "updateChecker": { "enabled": true, "channel": "stable" }
}
```

telemetry

관측 가능성

theme

UI 테마

editor

편집기 선호

outputStyle

응답 톤

# 병합 규칙

## 여러 계층이 충돌할 때

| 필드                | 병합 방식                  | 예시                          |
|-------------------|------------------------|-----------------------------|
| permissions.deny  | 모든 계층의 union (deny 우선) | managed + project + user 합산 |
| permissions.allow | 상위 계층 우선 (overwrite)   | managed가 있으면 그것만 적용         |
| permissions.ask   | 상위 계층 우선               | managed > project > user    |
| model             | 상위 계층 우선               | managed 강제 시 user 무시        |
| hooks             | 모든 계층 concat (배열 합산)   | 여러 hook이 모두 실행              |
| mcpServers        | 키 단위 병합 (이름 같으면 상위 우선) | 같은 이름은 managed가 이김          |

**deny wins**

deny 우선

**higher wins**

상위 우선

**arrays concat**

배열 합산

**objects merge**

객체 병합

# 완성 예시 1 / 개인 사용자

~/.claude/settings.json

personal

```
# ~/.claude/settings.json
{
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob", "WebFetch",
      "Bash(git diff:*)", "Bash(git log:*)",
      "Bash(npm test:*)", "Bash(npm run:*)"
    ],
    "ask": ["Edit", "Write", "Bash(git push:*)"]
  },
  "model": "claude-sonnet-4-5",
  "modelAlias": {
    "fast": "claude-haiku-4-5",
    "smart": "claude-opus-4-5"
  },
}
```

```
"theme": "dark",
"outputStyle": "concise",
"editor": {
  "tabWidth": 2,
  "preferredQuotes": "single"
},
"mcpServers": {
  "filesystem": {
    "command": "npx",
    "args": ["-y", "@modelcontextprotocol/server-filesystem",
      "/Users/me/Documents"]
  }
}
}
```

Personal

개인 전역

Liberal

관대한 권한

Sonnet

균형 모델

filesystem

기본 MCP



# 완성 예시 2 / 팀 프로젝트

## .claude/settings.json

```
# .claude/settings.json (Git 커밋, 팀 공유)
{
  "permissions": {
    "allow": [
      "Read(src/**)", "Read(tests/**)", "Read(docs/**)",
      "Edit(src/**)", "Edit(tests/**)", "Edit(docs/**)",
      "Bash(npm test:*)", "Bash(npm run lint)",
      "Bash(npm run build)"
    ],
    "ask": [
      "Write(**)", "Bash(git push:*)", "Bash(git commit:*)"
    ],
    "deny": [
      "Read(**/.env*)", "Read(**/secrets/**)",
      "Bash(rm -rf:*)", "Bash(curl:*)"
    ]
  },
}
```

```
    "hooks": {
      "PostToolUse": [
        {
          "matcher": "Write|Edit",
          "hooks": [{
            "type": "command",
            "command": "npm run lint -- --fix"
          }]
        }
      ]
    },
    "mcpServers": {
      "github": {
        "command": "npx",
        "args": ["-y", "@modelcontextprotocol/server-github"]
      }
    }
  }
}
```

team

Project

팀 공유

Scoped

경로 제한

Auto-fix

lint 자동

GitHub

MCP 통합

# 완성 예시 3 / 조직 표준

## /etc/claude/managed-settings.json

managed

```
# /etc/claude/managed-settings.json (root only, 강제)
{
  "permissions": {
    "deny": [
      "Bash(rm -rf:*)", "Bash(curl:*)", "Bash(wget:*)",
      "Bash(* > /etc/*)",
      "Read(**/.env*)", "Read(**/credentials/**)",
      "Read(**/private_key*)", "Read(**/.ssh/**)"
    ]
  },
  "model": "claude-sonnet-4-5",
  "modelAlias": {
    "smart": "claude-sonnet-4-5" # smart도 Sonnet (Opus 차단)
  },
  "deniedModels": ["claude-opus-4-5"],
```

```
"hooks": {
  "PreToolUse": [
    {
      "matcher": "Bash|Edit|Write",
      "hooks": [{ "type": "command", "command": "/opt/claude/dlp.sh" }]
    }
  ],
  "SessionStart": [
    { "hooks": [{ "type": "command", "command": "/opt/claude/policy.sh" }] }
  ]
},
"telemetry": {
  "enabled": true,
  "endpoint": "https://otel.company.com:4318"
},
"updateChecker": { "enabled": false }
}
```

**Mandatory**

강제 적용

**No Opus**

비용 통제

**DLP Hook**

보안 강제

**Telemetry**

감사 강제

# settings.local.json

## 개인의 프로젝트별 오버라이드

### USE CASE

settings.local.json은 프로젝트 .claude/ 디렉토리에 있지만 Git에 커밋하지 않는 파일입니다. 본인만의 프로젝트별 설정을 덮어쓸 때 사용합니다.

```
# .gitignore에 추가
.claude/settings.local.json

# .claude/settings.local.json (개인 오버라이드)
{
  "permissions": {
    "allow": [
      "Bash(npm run dev:*)",
      "Read(/Users/me/notes/**)"
    ]
  },
}
```

```
"mcpServers": {
  "my-personal-jira": {
    "command": "npx",
    "args": ["-y", "@my-org/jira-mcp"],
    "env": { "JIRA_TOKEN": "my-personal-token" }
  }
}
}
```

local override

# 효과

- 팀 설정은 그대로 유지
- 본인만의 권한과 MCP 추가 가능
- 다른 팀원에게 영향 없음

Git-ignored

커밋 안 함

Personal

본인 전용

Project-scoped

프로젝트 한정

Additive

추가만 가능

# Settings 검증

## 오류 진단과 디버깅

### Validation

# 1. claude doctor로 모든 설정 검증

```
$ claude doctor
```

Settings:

Resolved: ~/.claude/settings.json + .claude/settings.json

Permissions: 12 allow, 3 ask, 5 deny rules ✓

Hooks: 2 PreToolUse, 1 PostToolUse ✓

MCP servers: filesystem (stdio), github (stdio) ✓

Telemetry: enabled, endpoint: https://otel.company.com:4318 ✓

# 2. JSON 문법 오류 시 직접 확인

```
$ jq . ~/.claude/settings.json
```

# JSON parse error 메시지로 라인 번호 표시

# 3. 권한 충돌 확인

```
> /permissions
```

# 현재 적용된 모든 권한 규칙 표시

# 각 규칙이 어느 계층에서 왔는지 함께 표시

# 4. Hooks 동작 확인

```
$ claude --debug
```

# hook 실행 시 stderr에 로그 출력

# 어느 hook이 어떤 입력을 받고 어떤 결과를 냈는지

# 5. MCP 서버 연결 확인

```
> /mcp
```

# 등록된 MCP 서버 목록과 연결 상태

doctor

종합 진단

jq

JSON 검증

/permissions

권한 확인

--debug

Hook 추적

# Part 1 Recap

## settings.json 구조 핵심 정리

### TAKEAWAY 1

#### 3단 계층

managed (강제) >  
project (팀) > user (개인)  
+ local (개인 오버라이드)

### TAKEAWAY 2

#### 6대 카테고리

permissions, model,  
hooks, mcp, telemetry,  
theme 모든 옵션 활용

### TAKEAWAY 3

#### 병합 규칙

deny는 union,  
allow/model은 상위 우선,  
hooks는 배열 concat

### TAKEAWAY 4

#### 실전 패턴

개인/팀/조직 3가지 레벨,  
settings.local로 개인 오버  
라이드

### NEXT - PART 2

Part 2는 Permissions 심화입니다. allow/ask/deny 패턴, 그룹, 명령 패턴, 권한 디버깅을 15장에 걸쳐 다룹니다.

Part 1

완료

Part 2

Permissions (15p)

Part 3

Hooks (15p)

~3.5h

잔여 시간

## CHAPTER 04 – PART 02

# Settings – Permission 심화

settings.json, Hooks, MCP 심화



# Part 2 / Permissions 심화

## 도구 권한 완전 정복

### PART OBJECTIVE

allow, ask, deny 3가지 규칙의 작성법, 그룹 패턴, 명령 패턴, 우선순위 결정, 권한 디버깅까지 Permissions를 완전히 마스터합니다.

#### TOPIC 1

##### 3가지 규칙

allow (자동), ask (확인),  
deny (차단) 각각의 정확한  
동작

#### TOPIC 2

##### 패턴 문법

도구 이름, 경로 그룹, 명령  
패턴, 와일드카드, 부정

#### TOPIC 3

##### 우선순위

구체적 패턴 우선, 충돌 해결  
, 다층 계층 병합

#### TOPIC 4

##### 디버깅

/permissions 명령, 적용  
흐름 추적, 일반 실수 회피

15 Slides

Part 2 분량

~30 min

예상 강의 시간

3 rules

권한 규칙

Master

학습 목표

# allow / ask / deny 차이

## 각 규칙의 정확한 동작

### ALLOW

#### 자동 허용

사용자 확인 없이 즉시 도구 실행. 안전하다고 판단된 도구에 적용. 가장 빈번한 규칙.

### ASK

#### 사용자 확인

도구 실행 전 사용자에게 yes/no 확인. 그레이존 작업에 적용. 한 번 승인 시 세션 동안 유효.

### DENY

#### 강제 차단

어떤 경우에도 실행 불가. 사용자가 명령해도 차단. 위험한 작업에 필수.

### GENERAL RULE

대부분의 작업은 allow로 자동 진행, 영향이 큰 작업은 ask로 확인, 명백한 위험은 deny로 차단.  
ask가 과도하면 사용자 피로도가 높아지고, ask가 부족하면 보안 사고 위험이 커짐.

### Auto

allow 동작

### Confirm

ask 동작

### Block

deny 동작

### Balance

균형 중요



# 패턴 문법

## 다양한 패턴 표현 방법

syntax

```
# 1. 도구 이름만 - 도구 전체 허용
"Read", "Grep", "Glob", "Edit", "Write"

# 2. 경로 글롭 - 도구 + 경로 제한
"Read(src/**)"      # src 하위 모든 파일
"Read(**/*.md)"     # 모든 .md 파일
"Edit(docs/**)"     # docs 디렉토리만
"Read(src/**, tests/**)" # 여러 경로 콤마 구분

# 3. Bash 명령 패턴
"Bash(npm test:*)"  # npm test로 시작하는 모든 명령
"Bash(npm test)"    # 정확히 'npm test' 만
"Bash(git diff:*)"  # git diff 변형들
"Bash(git status)"  # 정확한 명령

# 4. 다중 명령 패턴
"Bash(npm test:*, npm run lint, npm run build)"

# 5. 부정 (deny에서 의미)
"Read(**/.env*)"    # .env 파일 차단
"Bash(*sudo*)"      # sudo 포함 명령 차단
"Read(/etc/**)"     # 시스템 디렉토리 차단

# 6. 와일드카드 (주의)
"Bash(*)"           # 모든 Bash 허용 (위험!)
"Edit(*)"           # 모든 Edit 허용 (주의)
```

Tool  
도구만

Path glob  
경로 패턴

Cmd pattern  
명령 패턴

Wildcard  
와일드카드

# Bash 패턴 심화

## 명령 화이트리스트 작성법

### Bash patterns

```
# 1. 정확 매칭 (가장 안전)
"Bash(npm test)"          # 'npm test' 이 명령만
"Bash(git status)"        # 'git status' 이 명령만

# 2. 점두사 매칭 (':*' 사용)
"Bash(npm test:*)"        # 'npm test --watch' 등 모두
"Bash(git diff:*)"        # 'git diff', 'git diff HEAD~1' 등

# 3. 부분 패턴 (정규식 비슷)
"Bash(* > /tmp/*)"        # /tmp로의 리디렉션만
"Bash(*sudo*)"            # sudo 포함 명령 (deny에서)

# 4. 자주 쓰는 안전한 패턴 (allow)
"Bash(npm test:*)",
"Bash(npm run lint)",
"Bash(npm run build)",
"Bash(npm run dev:*)",
"Bash(git status)",
"Bash(git diff:*)",
"Bash(git log:*)",
"Bash(git show:*)",
"Bash(ls:*)",
"Bash(pwd)"

# 5. 위험 패턴 (deny에 추가)
"Bash(rm -rf:*)",
"Bash(rm -f:*)",
"Bash(sudo:*)",
"Bash(chmod:*)",
"Bash(* > /etc/*)",
"Bash(curl:*)", "Bash(wget:*)"
```

Exact

정확 매칭

:\*

점두사

Safe list

안전 목록

Deny list

차단 목록

# 경로 글롭 패턴

## 디렉토리와 파일 제한

glob patterns

```
# 1. 단일 디렉토리
"Read(src/**)"          # src 하위 모든
"Read(src/api/**)"      # src/api 하위만
"Read(./README.md)"     # 정확한 파일

# 2. 파일 확장자 패턴
"Read(**/*.md)"          # 모든 마크다운
"Edit(**/*.{js,ts})"     # JS와 TS 파일
"Read(**/*.{yaml,yml})" # YAML 파일

# 3. 여러 경로 조합
"Read(src/**, tests/**, docs/**)"
"Edit(src/**)"           # 작성은 src만
```

```
# 4. 제외 패턴 (deny와 조합)
allow: "Read(**)"
deny:  "Read(**/.env*)" # .env 파일은 제외
deny:  "Read(**/node_modules/**)"

# 5. 절대 경로
"Read(/Users/me/shared/**)" # 홈 외부 디렉토리
"Read(/etc/claude/**)"      # 시스템 디렉토리 (특별)

# 6. 현재 프로젝트 외부 접근
# settings.json의 additionalDirectories
{
  "permissions": {
    "additionalDirectories": ["/Users/me/shared-lib"],
    "allow": ["Read(/Users/me/shared-lib/**)"]
  }
}
```

**/\*\***

재귀 매칭

**\*.ext**

확장자

**{a,b}**

여러 옵션

**additional**

외부 디렉토리

# 구체성 우선 규칙

## 충돌 시 어느 패턴이 이기는가

# 시나리오: 두 규칙이 모두 매칭될 때

specificity

# 예시 1: allow와 deny 충돌

allow: "Bash(\*)" # 모든 Bash 허용

deny: "Bash(rm -rf:\*)" # rm -rf 차단

→ 결과: rm -rf는 차단 (deny가 더 구체적이고 deny 우선)

# 예시 2: 두 allow 충돌

allow: "Read(\*\*)" # 모두 허용

allow: "Read(src/\*\*)" # src만 명시

→ 결과: 둘 다 적용 (allow끼리는 더 관대한 쪽 적용)

# 예시 3: allow와 ask 충돌

allow: "Edit(docs/\*\*)" # docs는 자동

ask: "Edit(\*\*)" # 일반 Edit은 확인

→ 결과: docs는 자동, 다른 경로는 ask

# 일반 규칙 (3가지)

1. deny > ask > allow (deny가 최우선)

2. 더 구체적인 패턴 > 일반 패턴

3. 더 상위 계층 > 하위 계층 (managed > project > user)

# 디버깅: /permissions 명령으로 어느 규칙이 적용되었는지 확인

deny first

1순위

specific

2순위

higher layer

3순위

/permissions

디버깅

# /permissions 명령

## 실시간 권한 확인과 디버깅

```
# 1. 모든 권한 규칙 표시
> /permissions
```

Current effective permissions:

ALLOW (12 rules):

|                    |                         |
|--------------------|-------------------------|
| ✓ Read             | [managed-settings.json] |
| ✓ Grep             | [managed-settings.json] |
| ✓ Glob             | [managed-settings.json] |
| ✓ Bash(npm test:*) | [.claude/settings.json] |
| ✓ Bash(git diff:*) | [.claude/settings.json] |
| ✓ Edit(docs/**)    | [.claude/settings.json] |
| ...                |                         |

ASK (3 rules):

|                    |                         |
|--------------------|-------------------------|
| ? Edit(src/**)     | [.claude/settings.json] |
| ? Write(**)        | [.claude/settings.json] |
| ? Bash(git push:*) | [.claude/settings.json] |

DENY (5 rules):

|                  |                         |
|------------------|-------------------------|
| x Bash(rm -rf:*) | [managed-settings.json] |
| x Bash(curl:*)   | [managed-settings.json] |
| x Read(**/.env*) | [managed-settings.json] |
| ...              |                         |

/permissions

```
# 2. 특정 도구 확인
```

```
> /permissions Bash(rm -rf foo)
→ DENY (matched by 'Bash(rm -rf:*)' from managed-settings.json)
```

/permissions  
명령

Source layer  
출처 표시

Specific check  
특정 도구

Live  
실시간

# 일반적인 실수

## 권한 설정 시 흔한 오류

### MISTAKE 1

#### 와일드카드 남용

Bash(\*) 같은 광범위 허용은 위험.  
구체적 명령으로 좁히세요.

### MISTAKE 2

#### 그룹 미사용

Read만 명시. src 외부도 접근 가능.  
Read(src/\*\*) 권장.

### MISTAKE 3

#### 경로 오타

docs/\*\* 대신 doc/\*\*. 매칭 안 되어 의도와 다르게 동작.

### MISTAKE 4

#### deny 누락

rm이나 curl 같은 위험 명령 deny 누락.  
사고 위험.

### CHECKLIST

권한 설정 후 반드시 /permissions로 검증. 의도한 도구가 의도한 동작으로 분류되는지 확인.  
새 권한 추가 후 위험 명령 시도로 deny 동작 확인.

#### Wildcard

남용 주의

#### Glob

경로 좁힘

#### Typo

오타 검증

#### Deny

위험 차단

# 세밀한 권한 설계

## 역할별 차등 적용

Role-based

# 시나리오: 신입 개발자 vs 시니어 개발자

# 신입 개발자 (보수적)

```
{
  "permissions": {
    "allow": [
      "Read(src/**)", "Read(tests/**)", "Read(docs/**)",
      "Grep", "Glob",
      "Bash(npm test:*)", "Bash(npm run lint)"
    ],
    "ask": [
      "Edit(src/**)", "Edit(tests/**)",
      "Write(**)", "Bash(git commit:*)"
    ],
    "deny": [
      "Edit(.github/**)", # CI 설정 변경 금지
      "Edit(scripts/**)", # 빌드 스크립트 변경 금지
      "Bash(npm run deploy:*)",
      "Bash(git push:*)"
    ]
  }
}
```

# 시니어 개발자 (관대)

```
{
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob",
      "Edit(src/**)", "Edit(tests/**)", "Edit(docs/**)",
      "Bash(npm:*)", "Bash(git:*)"
    ],
    "ask": ["Write(**)", "Bash(git push:*)"],
    "deny": ["Bash(rm -rf:*)", "Bash(sudo:*)"]
  }
}
```

Junior

보수적

Senior

관대

Role-based

역할 기반

Promotion

승급 경로

# 민감 데이터 보호

## deny 패턴으로 사고 예방

sensitive deny

# 반드시 deny에 포함해야 할 패턴들

# 1. 자격증명 파일

```
"Read(**/.env*)",  
"Read(**/.env.local)",  
"Read(**/credentials)",  
"Read(**/.aws/credentials)",  
"Read(**/.aws/config)",
```

# 2. SSH와 GPG 키

```
"Read(**/.ssh/**)",  
"Read(**/id_rsa)", "Read(**/id_ed25519)",  
"Read(**/.gnupg/**)",
```

# 3. 클라우드 자격증명

```
"Read(**/.kube/config)",  
"Read(**/gcloud/**)",
```

# 4. 사내 시크릿

```
"Read(**/secrets/**)",  
"Read(**/secret*)",  
"Read(**/private_key*)",  
"Read(**/*.pem)", "Read(**/*.key)",
```

# 5. 위험 시스템 경로

```
"Read(/etc/shadow)",  
"Read(/etc/sudoers)",  
"Read(/proc/**)",  
"Edit(/etc/**)",
```

# 6. 데이터베이스 덤프

```
"Read(**/*.sql.gz)",  
"Read(**/backup/**)"
```

Credentials

자격증명

Keys

SSH/GPG

Secrets

시크릿

System

시스템 파일



# additionalDirectories

## 프로젝트 외부 접근 허용

# 기본 동작

Claude Code는 현재 작업 디렉토리(cwd) 외부 접근을 차단합니다.  
홈 디렉토리나 시스템 디렉토리는 기본적으로 못 들어갑니다.

# 외부 디렉토리 허용 방법

```
{
  "permissions": {
    "additionalDirectories": [
      "/Users/me/shared-libs",
      "/Users/me/personal-notes",
      "~/.config/myapp"
    ],
    "allow": [
      "Read(/Users/me/shared-libs/**)",
      "Read(/Users/me/personal-notes/**)"
    ]
  }
}
```

additionalDirs

# 주의 사항

1. additionalDirectories로 추가만 가능하지 권한이 자동 부여 X
2. allow에도 명시적으로 경로 패턴 추가 필요
3. 절대 경로 또는 홈 디렉토리 expansion (~) 가능

# 보안 권장

- 가능한 한 좁은 디렉토리만 추가
- /Users/me 같은 홈 전체는 절대 피하기
- 추가 후 deny 패턴으로 민감 파일 명시 차단

cwd-only

기본 제한

Add explicit

명시 추가

allow needed

권한 별도

Scope tight

좁게

# Ask 승인 흐름

## 사용자 확인 UX

### STEP 01

Claude가 도구  
호출 시도

### STEP 02

ask 규칙  
매칭 확인

### STEP 03

사용자에게  
확인 요청

### STEP 04

사용자  
승인 또는 거부

### STEP 05

도구 실행  
또는 중단

# 사용자에게 표시되는 확인 메시지 예시

> Edit src/payment.js

Allow this Edit operation?

File: src/payment.js

Diff preview:

```
- function processPayment(amount) {  
+ function processPayment(amount, currency = 'USD') {  
+   if (!amount || amount <= 0) throw new Error(...);  
deny 추가됨
```

```
[y] Yes, allow once  
[a] Yes, allow this tool for session  
[n] No, deny this time  
[d] No, and add to deny list
```

Your choice:

# 세션 동안 'a' 선택 시 같은 도구는 자동 허용  
# 'd' 선택 시 settings.local.json에

approval UX

Visible diff

변경 미리보기

4 options

응답 옵션

Per-session

세션 단위

Persistent

영구 차단

# Permissions vs Hooks

## 두 메커니즘의 차이

### PERMISSIONS

#### 권한 통제

1. 정적 규칙 (선언적)
2. 도구 호출 전 매칭 검사
3. allow/ask/deny 세 가지
4. settings.json에 정의
5. 빠른 검사 (오버헤드 적음)
6. 패턴 기반 (그룹, 명령 패턴)

### HOOKS

#### 동적 제어

1. 동적 스크립트 (절차적)
2. 도구 호출 전후 실행
3. 임의 로직 가능 (DLP, 로깅)
4. settings.json + 외부 스크립트
5. 느린 검사 (스크립트 실행)
6. 컨텍스트 기반 (조건 판단)

Static

Permissions

Dynamic

Hooks

Fast

오버헤드

Combine

함께 사용

# Permissions Best Practices

## 실전 권장 사항 6가지

### BP 1

#### 최소 권한

필요한 것만 allow, 나머지는 기본 ask.  
광범위 와일드카드 회피.

### BP 2

#### 경로 좁힘

Read 별 대신 Read(src/별  
별)처럼 디렉토리 명시.  
외부 접근 차단.

### BP 3

#### Deny 우선 설계

위험 패턴은 deny에 먼저  
추가.  
allow는 그 후에 적절히.

### BP 4

#### 계층별 분리

managed는 deny 위주,  
project는 allow 위주,  
user는 개인 선호.

### BP 5 / 정기 리뷰

권한 정책은 한 번 만들고 끝이 아니며, 분기마다 검토하고 ask 승인율을 보고 조정.  
항상 승인되는 ask는 allow로, 거의 거부되는 ask는 deny로.

#### Least-priv

최소 권한

#### Scoped

경로 좁힘

#### Deny-first

Deny 우선

#### Review

정기 검토

# Part 2 Recap

## Permissions 핵심 정리

### TAKEAWAY 1

#### 3가지 규칙

allow (자동) / ask (확인) /  
deny (차단), deny가 항상  
승리

### TAKEAWAY 2

#### 패턴 문법

도구 이름, 경로 그룹, Bash  
명령 패턴, 다중 패턴 조합

### TAKEAWAY 3

#### 우선순위

deny > ask > allow, 구체  
적 > 일반적, 상위 계층 > 하  
위

### TAKEAWAY 4

#### BP & 디버깅

최소 권한, /permissions  
로 검증, 정기 리뷰로 진화

### NEXT - PART 3

Part 3은 Hooks 4가지 시점입니다. PreToolUse, PostToolUse, SessionStart, Stop 각각의 동작과 활용 패턴을 15장에 걸쳐 다룹니다.

### Part 2

완료

### Part 3

Hooks (15p)

### Part 4

Hook Patterns (15p)

### ~3h

잔여 시간

CHAPTER 04 – PART 03

# Settings – Hooks

settings.json, Hooks, MCP 심화



# Part 3 / Hooks 4가지 시점

## 동적 자동화의 핵심 메커니즘

### PART OBJECTIVE

PreToolUse, PostToolUse, SessionStart, Stop 4가지 Hook 시점의 정확한 동작, 입출력 스키마, 매치 패턴을 학습합니다.

#### HOOK 1

##### PreToolUse

도구 호출 직전. 차단 가능.  
DLP, 정책 검증, 외부 알림  
에 사용.

#### HOOK 2

##### PostToolUse

도구 호출 직후. 결과 가공.  
자동 lint, 포맷, 후처리에 활  
용.

#### HOOK 3

##### SessionStart

세션 시작 시 한 번. 환경 초  
기화, 정책 안내, 컨텍스트  
주입.

#### HOOK 4

##### Stop

Claude 응답 종료 시. 정리,  
알림 발송, 메트릭 집계에 활  
용.

15 Slides

Part 3 분량

~30 min

예상 강의 시간

4 hooks

시점 종류

Dynamic

동적 제어

# Hook 구조

## matcher와 command

### Hook structure

```
# 일반 구조
{
  "hooks": {
    "<hook-type>": [
      {
        "matcher": "<pattern>",    // 어느 도구에 적용
        "hooks": [
          {
            "type": "command",
            "command": "<script-path>",
            "timeout": 5000      // 옵션: ms 단위
          }
        ]
      }
    ]
  }
}
```

```
# 실제 예시
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|Edit|Write",  // 정규식
        "hooks": [
          {
            "type": "command",
            "command": "/opt/claude/dlp-check.sh",
            "timeout": 3000
          }
        ]
      }
    ]
  }
}
```

```
# matcher 예시
# "Bash"          - Bash 도구만
# "Bash|Edit"     - Bash 또는 Edit
# ".*"           - 모든 도구
# "Write"         - Write만
```

matcher

대상 패턴

command

실행 스크립트

timeout

제한 시간

regex

정규식



# Hook 입력 형식

## stdin으로 전달되는 JSON

# Hook 스크립트는 stdin으로 JSON 받음

# PreToolUse 입력 예시

```
{
  "session_id": "ses_abc123",
  "tool_name": "Bash",
  "tool_input": {
    "command": "npm test",
    "description": "Run test suite"
  }
}
```

# Edit 도구 호출 시

```
{
  "session_id": "ses_abc123",
  "tool_name": "Edit",
  "tool_input": {
    "file_path": "/path/to/file.js",
    "old_string": "...",
    "new_string": "..."
  }
}
```

input format

# 스크립트에서 파싱 (bash + jq)

#!/bin/bash

INPUT=\$(cat)

TOOL=\$(echo "\$INPUT" | jq -r '.tool\_name')

COMMAND=\$(echo "\$INPUT" | jq -r '.tool\_input.command // empty')

FILE=\$(echo "\$INPUT" | jq -r '.tool\_input.file\_path // empty')

```
if [ "$TOOL" = "Bash" ] && echo "$COMMAND" | grep -q 'rm -rf'; then
  echo '{"action":"block","reason":"위험 명령"}' >&2
  exit 1
fi
```

```
exit 0
```

stdin

입력 통로

JSON

구조화

session\_id

세션 식별

jq

파싱 도구

# Hook 출력 형식

## 차단과 통과 결정

output format

```
# Hook 출력은 exit code와 stderr JSON으로 결정
```

```
# 통과 (도구 호출 진행)
```

```
exit 0
```

```
# stderr에 아무것도 출력 안 함
```

```
# 차단 (도구 호출 거부)
```

```
echo '{"action":"block","reason":"위험 명령 감지"}' >&2
```

```
exit 1
```

```
# 경고만 (도구는 진행, 사용자에게 알림)
```

```
echo '{"action":"warn","message":"이 명령은 위험할 수 있습니다"}' >&2
```

```
exit 0
```

```
# 입력 변환 (PreToolUse에서 가능)
```

```
echo '{
```

```
  "action":"modify",
```

```
  "tool_input": {
```

```
    "command": "npm test -- --silent"
```

```
  }
```

```
} ' >&2
```

```
exit 0
```

```
# 일반적인 패턴
```

```
case "$DECISION" in
```

```
  safe)    exit 0 ;;
```

```
  block)   echo '{"action":"block","reason":"..."}' >&2; exit 1 ;;
```

```
  warn)    echo '{"action":"warn","message":"..."}' >&2; exit 0 ;;
```

```
esac
```

```
# 비동기 외부 호출 (SIEM 전송 등)도 가능
```

```
# 단, timeout 내에 끝나야 함
```

exit 0

통과

exit 1

차단

action

JSON 액션

stderr

출력 통로

# PreToolUse 심화

## 도구 호출 직전 검증

### USE CASE 1

#### DLP 검사

민감 정보 입력 차단. AWS key, SSH key, 신용카드 번호 등 패턴 감지.

### USE CASE 2

#### 권한 추가 검증

permissions 규칙을 넘어 동적 조건 검증. 사용자 그룹, 시간대, 위치 기반.

### USE CASE 3

#### 입력 변환

도구 입력을 자동 수정. 환경별 다른 명령으로 변환 등.

### CRITICAL POINT

PreToolUse Hook이 가장 강력함. 도구 호출을 막을 수 있는 유일한 시점이며 organization 보안 정책의 핵심 메커니즘.

Block

차단 가능

Modify

입력 변환

Critical

보안 핵심

Fast

빠른 실행 필수

# PreToolUse 실전 예시

## DLP 스크립트 완성본

```
#!/bin/bash
# /opt/claude/hooks/pre-tool-use-dlp.sh

INPUT=$(cat)
TOOL=$(echo "$INPUT" | jq -r '.tool_name')

# Bash 명령 검사
if [ "$TOOL" = "Bash" ]; then
    CMD=$(echo "$INPUT" | jq -r '.tool_input.command')

    # 위험 명령 차단
    if echo "$CMD" | grep -qE 'rm -rf|chmod 777|sudo'; then
        echo '{"action":"block","reason":"위험 명령"}' >&2
        exit 1
    fi
fi
```

```
# Write/Edit 콘텐츠 검사
if [ "$TOOL" = "Write" ] || [ "$TOOL" = "Edit" ]; then
    CONTENT=$(echo "$INPUT" | jq -r \
        '.tool_input.content // .tool_input.new_string')

    # AWS Key 패턴
    if echo "$CONTENT" | grep -qE 'AKIA[0-9A-Z]{16}'; then
        echo '{"action":"block","reason":"AWS Key 감지"}' >&2
        exit 1
    fi

    # SSH Private Key
    if echo "$CONTENT" | grep -q 'BEGIN RSA PRIVATE KEY'; then
        echo '{"action":"block","reason":"SSH key 감지"}' >&2
        exit 1
    fi
fi

exit 0
```

DLP example

Bash check

명령 차단

Write check

콘텐츠 검사

Pattern

정규식

Production

실무 사용

# PostToolUse 심화

## 도구 호출 직후 처리

### USE CASE 1

#### 자동 포매팅

Write/Edit 후 prettier,  
black 같은 포맷터 자동 실행

### USE CASE 2

#### 린트 자동 적용

eslint --fix, ruff check --  
fix 같은 자동 수정

### USE CASE 3

#### 테스트 자동 실행

변경된 파일과 관련된 테스트  
만 자동 실행하여 즉시 검증

### USE CASE 4

#### 감사 로깅

모든 도구 호출 결과를  
SIEM 또는 로그 시스템에  
전송

### NOTE

PostToolUse는 차단할 수 없습니다. 도구는 이미 실행된 상태이므로 후처리만 가능합니다. 진행을 멈추려면 PreToolUse를 사용하세요.

#### Format

자동 포맷

#### Lint fix

자동 수정

#### Test

자동 검증

#### Log

감사 기록

# PostToolUse 실전 예시

## 자동 포맷 + lint fix

```
#!/bin/bash
# /opt/claude/hooks/post-tool-use-format.sh
```

```
INPUT=$(cat)
TOOL=$(echo "$INPUT" | jq -r '.tool_name')
```

```
# Write 또는 Edit 호출 후만 처리
if [ "$TOOL" != "Write" ] && [ "$TOOL" != "Edit" ]; then
    exit 0
fi
```

```
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path')
```

```
# 확장자별 포맷터 선택
case "$FILE" in
  *.js|*.jsx|*.ts|*.tsx)
    if command -v prettier >/dev/null; then
      prettier --write "$FILE" 2>/dev/null
      eslint --fix "$FILE" 2>/dev/null
    fi
    ;;
  *.py)
    if command -v black >/dev/null; then
      black "$FILE" 2>/dev/null
      ruff check --fix "$FILE" 2>/dev/null
    fi
    ;;
  *.go)
    gofmt -w "$FILE" 2>/dev/null
    ;;
  *.rs)
    rustfmt "$FILE" 2>/dev/null
    ;;
  *)
    ;;
esac

exit 0
```

auto format

Per-language

언어별 분기

Prettier+ESLint

JS/TS

Black+Ruff

Python

Silent fail

무중단

# SessionStart 심화

## 세션 시작 시 초기화

### USE CASE 1

#### 환경 검증

필수 환경변수, 자격증명, 도  
구 설치 상태 확인 후 안내

### USE CASE 2

#### 정책 안내 배너

사용 정책, 비용 한도, 금지  
사항을 매 세션 시작 시 표시

### USE CASE 3

#### 컨텍스트 주입

프로젝트 상태, 최근 git  
log, 사용자 정보를 자동 주  
입

### USE CASE 4

#### 버전 체크

Claude Code 버전 확인,  
업데이트 알림, 사내 표준 일  
치 확인

### EXECUTION

SessionStart는 매 세션 시작 시 한 번 실행. /clear 명령으로도 트리거되지 않으며 새 claude 호출 시에만 실행.

Once per session

실행 빈도

Init

초기화 목적

Inform

안내 목적

Validate

검증 목적

# SessionStart 실전 예시

## 정책 배너 + 환경 검증

session start

```
#!/bin/bash
# /opt/claude/hooks/session-start.sh

# 1. 정책 배너 출력 (사용자에게 표시)
cat <<EOF >&1
```

---

Claude Code - Company Internal Use

---

- ✓ 사용 활동은 사내 SIEM에 기록됩니다
- ✓ 민감 정보 입력은 자동 차단됩니다
- ✓ 비용 한도: 월 \$50/인 (초과 시 사용 제한)
- ✓ 정책 위반 시: security@company.com

허용 모델: Sonnet, Haiku (Opus는 별도 신청)  
허용 MCP: jira, confluence, gitlab, github

---

EOF

```
# 2. 환경 검증
if [ -z "$AWS_PROFILE" ]; then
    echo "△ AWS_PROFILE 미설정. SSO 로그인 필요." >&2
fi

if ! aws sts get-caller-identity >/dev/null 2>&1; then
    echo "△ AWS 자격증명 만료. aws sso login 실행." >&2
fi

# 3. 사용자 컨텍스트 주입 (stdout이 메인 프롬프트에 추가됨)
echo "## Session Context"
echo "User: $(whoami)"
echo "Project: $(basename $(pwd))"
echo "Git branch: $(git branch --show-current 2>/dev/null)"

exit 0
```

**Banner**  
정책 안내

**Validate**  
환경 검증

**Inject**  
컨텍스트 주입

**Every session**  
매 세션



# Stop Hook 심화

## 응답 종료 시 정리

### USE CASE 1

#### 세션 메트릭 집계

토큰 사용량, 도구 호출 수,  
비용 등 세션 통계를 외부로  
전송

### USE CASE 2

#### 임시 파일 정리

세션 중 생성된 임시 파일이  
나 캐시 파일을 자동 삭제

### USE CASE 3

#### 작업 알림

오래 걸린 작업 완료 후  
Slack, 이메일, 데스크톱 알  
림 전송

### USE CASE 4

#### 컨텍스트 저장

세션 요약을 파일로 저장해  
다음 세션에 재활용

### TIMING

Stop은 Claude가 응답을 완료하고 다음 사용자 입력을 기다리는 시점에 실행. 세션 종료(/exit)와는 다른 시점이니 주의.

### Metrics

메트릭

### Cleanup

정리

### Notify

알림

### Save

저장

# Stop 실전 예시

## Slack 작업 완료 알림

stop notify

```
#!/bin/bash
# /opt/claude/hooks/stop-notify.sh

INPUT=$(cat)
SESSION_ID=$(echo "$INPUT" | jq -r '.session_id')

# 환경변수에서 Slack webhook URL 가져옴
if [ -z "$SLACK_WEBHOOK_URL" ]; then
    exit 0
fi

# 세션 소요 시간 계산 (예시: 5분 이상이면 알림)
SESSION_FILE="/tmp/claude-session-$SESSION_ID"
if [ ! -f "$SESSION_FILE" ]; then
    date +%s > "$SESSION_FILE"
    exit 0
fi

START=$(cat "$SESSION_FILE")
NOW=$(date +%s)
DURATION=$((NOW - START))

if [ "$DURATION" -lt 300 ]; then
    # 5분 미만은 알림 안 함
    exit 0
fi
```

```
# Slack 알림 발송
MIN=$((DURATION / 60))
USER=$(whoami)
PROJECT=$(basename "$(pwd)")

curl -s -X POST "$SLACK_WEBHOOK_URL" \
-H 'Content-Type: application/json' \
-d '{"text": "\ud83d\ude0a Claude task completed in $PROJECT (${MIN}min) by $USER\":" \
>/dev/null

exit 0
```

Slack  
알림 채널

>5min  
임계치

Async  
비동기

Webhook  
발송 방식

# Hook 체이닝

## 같은 시점에 여러 Hook

chaining

```
# 여러 Hook을 순차 실행
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|Edit|Write",
        "hooks": [
          { "type": "command", "command": "/opt/claude/dlp-check.sh" },
          { "type": "command", "command": "/opt/claude/audit-log.sh" },
          { "type": "command", "command": "/opt/claude/cost-check.sh" }
        ]
      }
    ]
  }
}
```

```
# 실행 순서
1. dlp-check.sh 실행
   - exit 10이면 → 차단 (다음 hook 실행 안 함)
   - exit 0이면 → 다음
2. audit-log.sh 실행
   - 항상 exit 0 (로깅만)
3. cost-check.sh 실행
   - 사용량 한도 초과 시 exit 1 차단
4. 모두 통과 시 도구 호출 진행

# matcher 별 분리도 가능
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Bash", "hooks": [...] },      # Bash 전용
      { "matcher": "Edit|Write", "hooks": [...] } # 쓰기 전용
    ]
  }
}
```

Sequential

순차 실행

Fail-fast

실패 시 중단

Per-matcher

매처별 분리

Composable

조합 가능

# Hook 디버깅

## 문제 진단과 검증

debug

```
# 1. --debug 모드로 hook 동작 추적
$ claude --debug
# stderr에 hook 실행 로그 출력
# 어느 hook이 어떤 입력을 받았는지, 어떤 결과를 냈는지

# 2. 수동 테스트 (Hook 스크립트 직접 실행)
$ echo '{
  "session_id": "test",
  "tool_name": "Bash",
  "tool_input": { "command": "rm -rf /tmp/test" }
}' | /opt/claude/hooks/dlp-check.sh

# 결과 확인
$ echo $? # exit code
1
$ # stderr에 '{"action": "block", "reason": "위험 명령"}' 출력
```

```
# 3. timeout 디버깅
# managed-settings에 timeout 충분히 설정
{
  "hooks": {
    "PreToolUse": [{
      "hooks": [{
        "type": "command",
        "command": "...",
        "timeout": 10000 # 10초로 늘려서 디버깅
      }]
    }]
  }
}
```

```
# 4. 로그 추가 (스크립트 내부)
echo "DEBUG: input=$INPUT" >> /tmp/claude-hook.log
echo "DEBUG: tool=$TOOL" >> /tmp/claude-hook.log
```

```
# 5. 일반 실수
- exit code 잘못: exit 1을 0으로 (차단이 안 됨)
- stderr 누락: JSON을 stdout으로 (Claude가 못 받음)
- jq 미설치: 명령 실행 실패
```

--debug

추적 모드

Manual test

수동 검증

Logging

내부 로그

jq required

의존성

# Part 3 Recap

## Hooks 4가지 시점 핵심 정리

### TAKEAWAY 1

#### 4가지 시점

PreToolUse (차단),  
PostToolUse (후처리),  
SessionStart (초기화),  
Stop (정리)

### TAKEAWAY 2

#### 입출력 규칙

stdin JSON 입력, stderr  
JSON + exit code로 결정  
전달

### TAKEAWAY 3

#### 체이닝

같은 시점 여러 hook 순차  
실행, fail-fast로 차단 시  
중단

### TAKEAWAY 4

#### 디버깅

--debug, 수동 테스트,  
timeout 늘림, 내부 로깅

### NEXT - PART 4

Part 4는 Hooks 실전 패턴입니다. 5가지 자동화 시나리오를 처음부터 끝까지 구현해 봅니다.

Part 3

완료

Part 4

Hook Patterns (15p)

Part 5-7

MCP + Commands (50p)

~2.5h

잔여 시간

CHAPTER 04 – PART 04

# Settings – Hooks 실전 패턴

settings.json, Hooks, MCP 심화



# Part 4 / Hooks 실전 패턴

## Hooks 4가지 시점 핵심 정리

### PATTERN 1

#### Auto Lint

---

Write/Edit 후 자동 lint와 포맷터 실행. 코드 일관성 강제.

### PATTERN 2

#### DLP Enforcement

---

민감 데이터 입력 차단. AWS key, SSH key, 신용카드 등 패턴.

### PATTERN 3

#### Audit Logging

---

모든 도구 호출 SIEM 전송. CloudTrail 또는 사내 시스템.

### PATTERN 4 & 5

Pattern 4 Slack Notify는 작업 완료 알림 전송. Pattern 5 Auto Format은 다양한 언어별 포맷터 자동 적용.

15 Slides

Part 4 분량

~30 min

예상 강의 시간

5 patterns

완성 코드

Copy-paste

즉시 활용

# Pattern 1 / Auto Lint

## 설계와 동작 방식

### STEP 01

Write/Edit  
도구 호출

### STEP 02

PostToolUse  
Hook 실행

### STEP 03

파일 확장자  
감지

### STEP 04

적절한  
linter 선택

### STEP 05

lint --fix  
자동 실행

### DESIGN GOAL

Claude가 코드를 작성하거나 수정할 때마다 즉시 팀 표준 lint를 자동 적용해 사람이 잊거나 PR에서 발견되는 일을 없앱니다.

| 언어     | Linter                | 명령                                  |
|--------|-----------------------|-------------------------------------|
| JS/TS  | ESLint + Prettier     | eslint --fix && prettier --write    |
| Python | Ruff + Black          | ruff check --fix && black           |
| Go     | gofmt + golangci-lint | gofmt -w && golangci-lint run --fix |

PostToolUse

Hook 시점

Per-language

언어별 분기

Auto-fix

자동 수정

Silent

무중단



# Pattern 1 / settings 등록

## managed-settings 또는 project 설정

settings reg

# 1. 프로젝트 .claude/settings.json

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "/opt/claude/hooks/auto-lint.sh",
            "timeout": 10000
          }
        ]
      }
    ]
  }
}
```

# 2. matcher 설명

- # "Write|Edit" - Write 또는 Edit 도구 호출 시 실행
- # "Edit" - Edit만
- # "Write" - Write만
- # "MultiEdit" - 여러 파일 동시 수정 (포함하려면 추가)

# 3. timeout 권장값

- # - lint만: 3000ms (3초)
- # - lint + 포맷: 5000ms
- # - 큰 프로젝트: 10000ms (10초)

# 4. 조직 표준화 (managed-settings)

- # /etc/claude/managed-settings.json에 동일 설정 두면
- # 모든 사용자의 모든 프로젝트에 적용됨

# 5. 환경변수로 도구 경로 전달

- # auto-lint.sh가 PATH에 의존하지 않게 환경변수 사용

Project  
프로젝트별

Managed  
조직 강제

5-10s  
timeout

Write|Edit  
matcher

# Pattern 1 / 스크립트 완성

## auto-lint.sh 전체 코드

```
#!/bin/bash
# /opt/claude/hooks/auto-lint.sh

INPUT=$(cat)
TOOL=$(echo "$INPUT" | jq -r '.tool_name')
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

# Write/Edit가 아니면 종료
[ "$TOOL" != "Write" ] && [ "$TOOL" != "Edit" ] && exit 0
[ -z "$FILE" ] && exit 0

# 파일이 존재하지 않으면 종료
[ ! -f "$FILE" ] && exit 0
```

```
# 확장자 추출
EXT="${FILE##*.*}"
case "$EXT" in
js|jsx|mjs|ts|tsx)
    if command -v prettier >/dev/null; then
        prettier --write "$FILE" 2>/dev/null
    fi
    if command -v eslint >/dev/null; then
        eslint --fix "$FILE" 2>/dev/null
    fi
    ;;
py)
    command -v black >/dev/null && black "$FILE" 2>/dev/null
    command -v ruff >/dev/null && ruff check --fix "$FILE" 2>/dev/null
    ;;
go)
    command -v gofmt >/dev/null && gofmt -w "$FILE" 2>/dev/null
    ;;
rs)
    command -v rustfmt >/dev/null && rustfmt "$FILE" 2>/dev/null
    ;;
json)
    command -v jq >/dev/null && \
        jq . "$FILE" > "${FILE}.tmp" && mv "${FILE}.tmp" "$FILE"
    ;;
esac
exit 0
```

auto-lint.sh

Per-ext  
확장자 분기

command -v  
도구 확인

Silent  
에러 숨김

Non-block  
항상 통과

# Pattern 2 / DLP Enforcement

## 민감 정보 차단 설계

### STEP 01

Bash/Edit/Write  
호출

### STEP 02

PreToolUse  
Hook 실행

### STEP 03

입력 콘텐츠  
패턴 검사

### STEP 04

민감 정보  
감지 여부

### STEP 05

차단 또는  
통과

### PATTERNS TO BLOCK

AWS Access Key, SSH Private Key, Database 연결 문자열, 신용카드 번호, 사내 시크릿 키워드 등 외부 노출 시 사고가 발생할 수 있는 모든 패턴.

#### 패턴

#### 정규식 예시

#### 심각도

AWS Access Key

AKIA[0-9A-Z]{16}

CRITICAL

SSH Private Key

BEGIN.\*PRIVATE KEY

CRITICAL

신용카드

WwB[0-9]{13,16}WwB

HIGH

### PreToolUse

Hook 시점

### Block

차단 액션

### Regex

검사 방식

### Critical

보안 핵심

# Pattern 2 / dlp.sh 완성

## DLP 검사 스크립트

dlp.sh

```
#!/bin/bash
# /opt/claude/hooks/dlp.sh

INPUT=$(cat)
TOOL=$(echo "$INPUT" | jq -r '.tool_name')

# 검사 대상 콘텐츠 추출
case "$TOOL" in
    Bash)
        CONTENT=$(echo "$INPUT" | jq -r '.tool_input.command') ;;
    Write)
        CONTENT=$(echo "$INPUT" | jq -r '.tool_input.content') ;;
    Edit)
        CONTENT=$(echo "$INPUT" | jq -r '.tool_input.new_string') ;;
    MultiEdit)
        CONTENT=$(echo "$INPUT" | jq -r '.tool_input.edits[].new_string') ;;
    *)
        exit 0 ;;
esac
```

```
# 패턴 검사 함수
check_pattern() {
    local pattern="$1"
    local reason="$2"
    if echo "$CONTENT" | grep -qE "$pattern"; then
        echo "{\"action\":\"block\",\"reason\":\"$reason\"}" >&2
        exit 1
    fi
}

# 민감 패턴 검사
check_pattern 'AKIA[0-9A-Z]{16}' 'AWS Access Key 감지'
check_pattern 'aws_secret_access_key.*.*[A-Za-z0-9/+]{40}' 'AWS Secret'
check_pattern 'BEGIN .* PRIVATE KEY' 'SSH/RSA Private Key 감지'
check_pattern 'sk-[a-zA-Z0-9]{32,}' 'API Key 패턴 감지'
check_pattern 'postgres://[^@]+@' 'DB 연결 문자열 감지'
check_pattern 'mongodb\+srv://[^@]+@' 'MongoDB 연결 감지'
check_pattern 'ghp_[a-zA-Z0-9]{36,}' 'GitHub Token 감지'

exit 0
```

Per-tool  
도구별 추출

Function  
검사 함수

7 patterns  
주요 패턴

Reusable  
재사용 가능

# Pattern 3 / Audit Logging

## 모든 활동 SIEM 전송

### STEP 01

도구 호출  
전후

### STEP 02

두 Hook 모두  
등록

### STEP 03

입력과 결과  
수집

### STEP 04

구조화 JSON  
페이로드

### STEP 05

SIEM API  
비동기 전송

### WHY BOTH PRE AND POST

PreToolUse만으로는 도구 호출 결과를 알 수 없습니다. PostToolUse도 등록해 실행 결과(성공/실패)까지 함께 기록해야 완전한 감사 추적  
이 가능.

```
# settings.json
{
  "hooks": {
    "PreToolUse": [{ "matcher": ".*", "hooks": [{ "type": "command", "command": "/opt/claude/audit.sh pre" }] }],
    "PostToolUse": [{ "matcher": ".*", "hooks": [{ "type": "command", "command": "/opt/claude/audit.sh post" }] } ]
  }
}
```

[register](#)

Pre + Post

양쪽 등록

All tools

. \* 매치

Async

비동기 전송

Hash content

본문 해시

# Pattern 3 / audit.sh 완성

## SIEM 전송 스크립트

```
#!/bin/bash
# /opt/claude/hooks/audit.sh
# Usage: audit.sh [pre|post]

PHASE="${1:-pre}"
INPUT=$(cat)

USER=$(whoami)
HOST=$(hostname)
TS=$(date -u +%Y-%m-%dT%H:%M:%SZ)

SESSION_ID=$(echo "$INPUT" | jq -r '.session_id')
TOOL=$(echo "$INPUT" | jq -r '.tool_name')
TOOL_INPUT=$(echo "$INPUT" | jq -c '.tool_input')
```

```
# 본문은 hash만 저장 (PII 보호)
CONTENT_HASH=$(echo -n "$TOOL_INPUT" | sha256sum | awk '{print $1}')

PAYLOAD=$(cat <<EOF
{
  "timestamp": "$TS",
  "phase": "$PHASE",
  "user": "$USER",
  "host": "$HOST",
  "session_id": "$SESSION_ID",
  "tool": "$TOOL",
  "tool_input_hash": "$CONTENT_HASH",
  "app": "claude-code"
}
EOF
)

# 사내 SIEM API로 비동기 전송 (백그라운드)
{
  curl -s -X POST "${SIEM_URL:-https://siem.company.com/api/log}" \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer ${SIEM_TOKEN}" \
    -d "$PAYLOAD" >/dev/null 2>&1
} &

exit 0
```

audit.sh

Hash

PII 보호

Background

비동기 &

Env vars

설정 분리

Non-block

항상 통과

# Pattern 4 / Slack Notify

## 작업 완료 알림 설계

### PROBLEM

복잡한 작업을 Claude에게 맡기고 다른 일을 하다가 작업이 끝난 줄 모르고 시간을 보내는 경우가 많습니다. Slack 알림으로 즉시 인지.

### TRIGGER

Stop Hook

Claude의 응답 종료 시점에  
trigger됨.

### CONDITION

시간 임계치

5분 이상 걸린 작업만 알림.  
짧은 응답은 노이즈가 됨.

### TARGET

Slack Channel

Webhook URL로 개인 채널  
또는 팀 채널에 전송 가능.

### CONTEXT

풍부한 정보

프로젝트명, 소요 시간, 사용자  
명, 마지막 메시지 요약 포함.

Stop

Hook 시점

>5min

임계치

Webhook

발송 방식

Context-rich

정보 포함

# Pattern 4 / slack-notify.sh

## 완성 스크립트

```
#!/bin/bash
# /opt/claude/hooks/slack-notify.sh
[ -z "$SLACK_WEBHOOK_URL" ] && exit 0
INPUT=$(cat)
SESSION_ID=$(echo "$INPUT" | jq -r '.session_id')
SESSION_FILE="/tmp/claude-session-$SESSION_ID"

# 첫 호출이면 시작 시간만 기록
if [ ! -f "$SESSION_FILE" ]; then
    date +%s > "$SESSION_FILE"
    exit 0
fi

START=$(cat "$SESSION_FILE")
NOW=$(date +%s)
DURATION=$((NOW - START))

# 5분 미만은 알림 안 함
[ "$DURATION" -lt 300 ] && exit 0

# 정보 수집
MIN=$((DURATION / 60))
USER=$(whoami)
PROJECT=$(basename "$(pwd)")
BRANCH=$(git branch --show-current 2>/dev/null || echo "no-git")
```

```
# Slack Block Kit 형식
PAYLOAD=$(cat <<EOF
{
  "blocks": [
    {
      "type": "section",
      "text": { "type": "mrkdwn",
        "text": "🎉 Claude task completed" }
    },
    {
      "type": "section",
      "fields": [
        { "type": "mrkdwn", "text": "*Project:*\\n$PROJECT" },
        { "type": "mrkdwn", "text": "*Branch:*\\n$BRANCH" },
        { "type": "mrkdwn", "text": "*User:*\\n$USER" },
        { "type": "mrkdwn", "text": "*Duration:*\\n${MIN} min" }
      ]
    }
  ]
}
EOF
)
curl -s -X POST "$SLACK_WEBHOOK_URL" \
  -H 'Content-Type: application/json' \
  -d "$PAYLOAD" >/dev/null 2>&1 &

# 다음 알림을 위해 시간 갱신
date +%s > "$SESSION_FILE"
exit 0
```

slack-notify.sh

Block Kit

Slack 포맷

Webhook

POST 발송

Reset

시간 갱신

Background

비동기



## CHAPTER 04 – PART 05

# Settings – Hooks 실전 패턴

settings.json, Hooks, MCP 심화



# Pattern 5 / Auto Format 완전판

## Multi-language + 사내 표준

```
#!/bin/bash
# /opt/claude/hooks/auto-format-pro.sh
# 사내 표준 도구 강제 + 다양한 언어 지원

INPUT=$(cat)
TOOL=$(echo "$INPUT" | jq -r '.tool_name')
[ "$TOOL" != "Write" ] && [ "$TOOL" != "Edit" ] && \
[ "$TOOL" != "MultiEdit" ] && exit 0

FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
[ -z "$FILE" ] || [ ! -f "$FILE" ] && exit 0

# 사내 표준 도구 경로 (PATH 우회)
COMPANY_BIN="/opt/company/bin"
```

```
# 프로젝트 root 찾기 (config 파일 우선)
ROOT=$(cd "$(dirname "$FILE")" && \
while [ ! -f .prettierrc ] && [ ! -f pyproject.toml ] && \
[ "$PWD" != "/" ]; do
    cd ..
done && pwd)
EXT="${FILE##*.}"
case "$EXT" in
js|jsx|mjs|cjs|ts|tsx)
    (cd "$ROOT" && "$COMPANY_BIN/prettier" --write "$FILE")
    (cd "$ROOT" && "$COMPANY_BIN/eslint" --fix "$FILE")
    ;;
py)
    (cd "$ROOT" && "$COMPANY_BIN/black" "$FILE")
    (cd "$ROOT" && "$COMPANY_BIN/ruff" check --fix "$FILE")
    ;;
go) gofmt -w "$FILE" ;;
rs) rustfmt --edition 2021 "$FILE" ;;
java) "$COMPANY_BIN/google-java-format" -i "$FILE" ;;
sql) "$COMPANY_BIN/sqlfluff" fix --dialect postgres "$FILE" ;;
esac
exit 0
```

auto-format-pro

Company bin

사내 표준

Root detect

config 위치

Multi-lang

6+ 언어

Production

실무 완성

# 패턴 조합

## 여러 Hook을 함께 사용

# managed-settings.json - 조직 표준 (모든 패턴 결합)

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash|Edit|Write|MultiEdit",
        "hooks": [
          { "type": "command", "command": "/opt/claude/hooks/dlp.sh" },
          { "type": "command", "command": "/opt/claude/hooks/audit.sh pre" }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [
          { "type": "command", "command": "/opt/claude/hooks/auto-format-pro.sh" }
        ]
      },
      {
        "matcher": ".*",
        "hooks": [
          { "type": "command", "command": "/opt/claude/hooks/audit.sh post" }
        ]
      }
    ]
  },
}
```

combined

```
"SessionStart": [
  { "hooks": [{ "type": "command", "command": "/opt/claude/hooks/session-start.sh" }] }
],
"Stop": [
  { "hooks": [{ "type": "command", "command": "/opt/claude/hooks/slack-notify.sh" }] }
]
}
```

# 실행 순서 (Write 호출 시)

1. PreToolUse: dlp.sh → audit.sh pre (둘 다 통과해야 진행)
2. Write 도구 실행
3. PostToolUse: auto-format-pro.sh → audit.sh post

All 5

5패턴 조합

Layered

계층 적용

Order

실행 순서

Standard

조직 표준

# 성능 고려사항

## Hook 오버헤드 관리

### TIP 1

#### Fast scripts

필수 검사만 동기 실행. 무거운 작업은 백그라운드 실행 (&)

### TIP 2

#### jq 한번만

여러 필드를 한 번의 jq 호출로 추출. 여러 번 호출 시 느림.

### TIP 3

#### Early return

관련 없는 도구는 즉시 exit 0. 불필요한 처리 회피.

### TIP 4

#### Timeout 적절히

PreToolUse는 3초 이내, PostToolUse는 10초 이내 권장

### MEASUREMENT

Hook 실행 시간을 측정하려면 스크립트 첫 줄에 `START=$(date +%s%N)`, 마지막에 `echo "$(((date +%s%N) - START) / 1000000))ms" >> /tmp/hook-perf.log`

#### Fast

빠른 동기

#### Background

느린 비동기

#### Early return

조기 종료

#### Measure

측정

# Hook 보안

## 스크립트 자체의 안전성

### RISK 1

#### Command injection

INPUT을 직접 명령에 넣지 마세요.  
항상 jq로 파싱 후 변수 인용.

### RISK 2

#### Path traversal

file\_path를 검증해 ../ 같은  
패턴 차단.  
realpath로 정규화.

### RISK 3

#### 권한 상승

sudo 사용 금지.  
필요 시 setuid 바이너리 별도  
설계.

### RISK 4

#### 비밀 노출

스크립트 안에 토큰 하드코딩  
금지.  
환경변수 사용.

### HARDENING

Hook 스크립트는 root 권한이거나 신뢰된 위치에 두어야 합니다. /opt/claude/hooks/는 root 소유로 두고 chmod 755로 설정하세요.  
사용자 수정 불가.

jq

안전 파싱

realpath

경로 정규화

No sudo

권한 상승 금지

Env vars

시크릿 분리

# Part 4 Recap

## Hooks 실전 패턴 핵심 정리

### TAKEAWAY 1

#### 5가지 완성 패턴

Auto Lint, DLP, Audit,  
Slack Notify, Auto  
Format - 그대로 활용

### TAKEAWAY 2

#### 조합 가능

5패턴 모두 동시 적용해 조  
직 표준 거버넌스 완성

### TAKEAWAY 3

#### 성능 의식

동기/비동기 분리, jq 한번,  
early return, timeout 적  
절히

### TAKEAWAY 4

#### 보안 고려

injection, path  
traversal, 권한 상승, 시크  
릿 노출 모두 대응

### NEXT - PART 5

Part 5는 MCP 기본과 공식 서버입니다. filesystem, github, slack, jira, postgres 5대 공식 서버를 20장에 걸쳐 다룹니다.

Part 4

완료

Part 5

MCP basics (20p)

Part 6

MCP custom (20p)

~2h

잔여 시간

# Part 5 / MCP 기본과 공식 서버

## Model Context Protocol 완전 이해

### PART OBJECTIVE

MCP 프로토콜의 동작 원리, 3가지 통신 방식, 5대 공식 서버 통합, 인증 설정, /mcp 명령 활용까지 MCP의 모든 기본기를 다룹니다.

#### TOPIC 1

##### MCP 프로토콜

JSON-RPC 기반,  
tools/resources/prompts  
3가지 primitive 개념

#### TOPIC 2

##### 3가지 통신

stdio (로컬), http (원격),  
sse (스트리밍) 각각의 사용  
시나리오

#### TOPIC 3

##### 5대 공식 서버

filesystem, github,  
slack, jira, postgres 등록  
과 활용

#### TOPIC 4

##### 운영 측면

인증 설정, /mcp 명령, 트러  
블슈팅, 보안 고려사항

20 Slides

Part 5 분량

~40 min

예상 강의 시간

5 servers

공식 서버

Foundation

기초 마스터

# MCP란 무엇인가

## Model Context Protocol 개요

### DEFINITION

MCP는 AI 모델이 외부 도구와 데이터에 표준화된 방식으로 접근하기 위한 오픈 프로토콜입니다. USB-C처럼 어떤 모델과 어떤 도구든 같은 규격으로 연결됩니다.

### WHY

#### 왜 필요한가

도구마다 다른 API를 직접 통합하면  $N \times M$  복잡도. MCP는 표준화로  $N + M$ 으로 줄임.

### WHO

#### 누가 만드는가

Anthropic이 주도하고 OpenAI, Google 등이 채택. 진정한 오픈 표준 추구.

### HOW

#### 어떻게 동작

JSON-RPC 2.0 프로토콜. Claude Code와 MCP 서버가 표준 메시지 교환.

Open

오픈 표준

JSON-RPC

프로토콜

$N+M$

복잡도 감소

Standard

표준화



# MCP의 3가지 Primitive

## Tools / Resources / Prompts

| Primitive | 용도                     | 예시                                     |
|-----------|------------------------|----------------------------------------|
| Tools     | AI가 호출하는 함수 (액션 수행)    | create_issue,<br>send_message          |
| Resources | AI가 읽는 데이터 (정적/동적 콘텐츠) | file:///path/to/doc.md,<br>db://schema |
| Prompts   | 재사용 가능한 프롬프트 템플릿       | "review-pr", "summarize-meeting"       |

# 예시: GitHub MCP 서버가 제공하는 것

# Tools (액션)

```
- create_issue(title, body, labels)
- merge_pr(pr_number, method)
- search_code(query, repo)
```

Tools

함수 호출

# Resources (데이터)

```
- github://repos/owner/repo/issues
- github://repos/owner/repo/pulls
```

Resources

데이터 접근

Prompts

템플릿

3 pillars

3대 요소

primitives

# 3가지 통신 방식

## stdio vs http vs sse

| Transport | 특징                           | 적합 시나리오           |
|-----------|------------------------------|-------------------|
| stdio     | 로컬 프로세스, Claude가 자식 프로세스로 실행 | 로컬 도구, 빠른 응답 필요   |
| http      | HTTP API 호출, 원격 서버           | 사내 마이크로서비스, 인증 필요 |
| sse       | Server-Sent Events, 양방향 스트리밍 | 장기 실행, 실시간 알림     |

|                             |        |      |        |              |
|-----------------------------|--------|------|--------|--------------|
| # settings.json - 3가지 방식 예시 |        |      |        | 3 transports |
| stdio                       | http   | sse  | Mix OK |              |
| 로컬                          | 원격 API | 스트리밍 | 혼합 사용  |              |

# stdio 통신 자세히

## 로컬 프로세스 실행 방식

stdio

```
# settings.json
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/me/Documents",
        "/Users/me/Projects"
      ],
      "env": {
        "LOG_LEVEL": "info"
      }
    }
  }
}
```

# 동작 흐름

1. Claude Code 시작 시 npx 명령 실행
2. MCP 서버가 자식 프로세스로 시작됨
3. stdin/stdout으로 JSON-RPC 메시지 교환
4. Claude Code 종료 시 자식 프로세스도 종료

# 장점

- 가장 빠른 응답 (네트워크 X)
- 별도 인프라 불필요
- 자격증명을 환경변수로 안전 전달

Fast  
응답

- 같은 머신에서만 동작  
- 서버 개수 불가 (각 Claude 인스턴스마다 별도)  
- 자원 소비 (각 서버마다 프로세스)

Local  
같은 머신

Env vars  
인증

Per-instance  
인스턴스별

# HTTP 통신 자세히

## 원격 MCP 서버

http

```
# settings.json
{
  "mcpServers": {
    "internal-mcp": {
      "type": "http",
      "url": "https://mcp.company.com/v1/mcp",
      "headers": {
        "Authorization": "Bearer ${MCP_TOKEN}",
        "X-Tenant-ID": "team-backend"
      },
      "timeout": 30000
    }
  }
}
```

- # 동작 흐름
1. Claude Code가 HTTP POST 요청
  2. body는 JSON-RPC 메시지
  3. 서버가 HTTP 응답으로 결과 반환
  4. 다음 호출은 새 HTTP 요청

- # 장점
- 중앙 집중식 운영 (한 서버, 여러 사용자)
  - 사내 인증 시스템과 통합 (Bearer, OAuth)
  - 모니터링과 로깅 용이
  - 업데이트 시 한 번에 전체 적용

- # 단점
- 네트워크 지연 (수십 ms)
  - 인프라 운영 부담

Centralized  
중앙 운영

Auth  
인증 통합

Network  
지연 있음

Multi-user  
다중 사용자

# SSE 통신 자세히

## Server-Sent Events 스트리밍

SSE

```
# settings.json
{
  "mcpServers": {
    "events-stream": {
      "type": "sse",
      "url": "https://mcp-events.company.com/sse",
      "headers": {
        "Authorization": "Bearer ${SSE_TOKEN}"
      }
    }
  }
}
```

### # 동작 흐름

1. Claude Code가 SSE 연결 시작 (long-lived HTTP)
2. 서버가 'data:' 라인으로 이벤트 스트림 전송
3. 양방향 메시지 교환 가능
4. 연결 유지 (재연결 자동)

### # 적합 시나리오

- 진행 상황 실시간 보고 (배포, 빌드 등)
- 외부 이벤트 푸시 (Slack 새 메시지, CI 결과)
- 장기 실행 작업의 중간 결과
- 양방향 대화형 워크플로

### # vs HTTP

- HTTP: 1요청 1응답 (짧은 작업)
- SSE: 1연결 N이벤트 (긴 작업, 푸시)

Long-lived

연결 유지

Bidirectional

양방향

Streaming

스트리밍

Real-time

실시간 푸시

# 공식 서버 1 / filesystem

## 로컬 파일 시스템 접근

filesystem

```
# 1. settings.json 등록
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/me/Documents",
        "/Users/me/Code",
        "/Users/me/Notes"
      ]
    }
  }
}
```

```
# 2. 제공되는 도구
- read_file(path)      - 파일 읽기
- write_file(path, content) - 파일 쓰기
- list_directory(path)  - 디렉토리 목록
- search_files(path, pattern) - 파일 검색
- move_file(source, dest) - 파일 이동
```

```
# 3. 사용 예시 (Claude에게)
> ~/Documents의 모든 .md 파일을 요약해 줘
```

```
# Claude가 자동으로 사용
```

1. list\_directory(~/Documents) 호출
2. .md 파일 필터링
3. 각 read\_file 호출
4. 요약 응답

Built in

공식 서버

- ```
# 4. 보안 주의
- 권한 부여한 디렉토리 외 접근 불가
- 다른 사용자 디렉토리는 명시 안 하면 막힘
```

5 tools

제공 도구

Path scoped

경로 한정

Common

가장 흔함

# 공식 서버 2 / github

## GitHub 통합

github

```
# 1. 토큰 발급
# https://github.com/settings/tokens → Generate new token
# 권한: repo (read+write), workflow, read:org
```

```
# 2. settings.json 등록
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${GH_TOKEN}"
      }
    }
  }
}
```

```
# 3. 제공되는 주요 도구
- create_issue(owner, repo, title, body)
- create_pull_request(owner, repo, title, head, base)
- get_pull_request(owner, repo, pr_number)
- merge_pull_request(owner, repo, pr_number)
- search_code(query)
- search_issues(query)
- get_file_contents(owner, repo, path)
- create_or_update_file(owner, repo, path, content)
```

```
# 4. 사용 예시
> 우리 organization의 모든 open PR을 요약해 줘
> 모든 변경사항으로 PR을 만들어 줘
```

PAI/App

인증 옵션  
- PAT (Personal Access Token) - 개인용  
- Github App - 조직용 (SSO 통합 가능)

PR/Issue  
주요 도구

Search  
검색

CI  
워크플로

# 공식 서버 3 / slack

## Slack 통합

slack

```
# 1. Slack App 생성 및 Bot Token 발급
# https://api.slack.com/apps → Create New App
# OAuth Scopes 추가:
#   - chat:write      (메시지 작성)
#   - channels:read   (채널 목록)
#   - users:read      (사용자 목록)
#   - search:read     (검색)

# 2. settings.json 등록
{
  "mcpServers": {
    "slack": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-slack"],
      "env": {
        "SLACK_BOT_TOKEN": "${SLACK_BOT_TOKEN}",
        "SLACK_TEAM_ID": "T0123456"
      }
    }
  }
}

# 3. 제공되는 도구
- send_message(channel, text)
- list_channels()
- list_users(channel)
- search_messages(query)
- get_thread_replies(channel, thread_ts)
- add_reaction(channel, ts, name)
```

### Bot Token

# 4. 사용 예시  
engineering 채널에 오늘 PR 머지 결과 요약 메시지  
이벤트 주 'urgent' 라벨된 메시지 찾아줘  
> 회의 결과를 #team-decisions에 정리해서 게시해줘

### Messaging

추천도

### Search

검색 가능

### Threads

스레드 지원



# 공식 서버 4 / postgres

## PostgreSQL 데이터 접근

postgres

```
# 1. settings.json 등록
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-postgres",
        "postgres://user:pass@host:5432/dbname"
      ]
    }
  }
}

# 또는 환경변수 사용 (권장)
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "POSTGRES_URL": "${DATABASE_URL}"
      }
    }
  }
}
```

```
# 2. 제공되는 도구
- query(sql)           - SQL 실행 (보통 read-only)
- list_tables()         - 테이블 목록
- describe_table(name) - 테이블 스키마
```

데이터 쿼리

Schema  
스키마 탐색

Read-only  
권장 권한

Replica  
복제본 사용

```
# 3. 사용 예제
> 어제 주문 건수를 시간대별로 보여줘
> users 테이블에 어떤 컬럼이 있어?
```

© 2024 Amazon.com, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

# 공식 서버 5 / jira

## JIRA 통합

jira

```
# 1. API 토큰 발급
# https://id.atlassian.com/manage-profile/security/api-tokens
# Create API Token

# 2. settings.json 등록
{
  "mcpServers": {
    "jira": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-jira"],
      "env": {
        "JIRA_URL": "https://company.atlassian.net",
        "JIRA_EMAIL": "user@company.com",
        "JIRA_API_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}

# 3. 제공되는 도구
- create_issue(project, summary, description)
- get_issue(issue_key)
- update_issue(issue_key, fields)
- search_issues(jql_query)
- add_comment(issue_key, body)
- transition_issue(issue_key, status)
- list_projects()
```

# 4. 사용 예시  
API Token  
PR의 변경 상태를 In Review로 변경해줘  
> 이번 스프린트의 모든 high 우선순위 버그를 찾아줘  
이 PR 내용으로 JIRA 티켓 생성해줘  
진행 중인 내 티켓들을 요약해줘

Issue CRUD  
티켓 관리

JQL  
복잡 검색

Workflow  
상태 전환

# /mcp 명령

## MCP 서버 상태 확인

/mcp

```
# 1. 현재 연결된 MCP 서버 목록
> /mcp
```

Connected MCP Servers:

```
filesystem      [stdio] ✓ Connected
Tools: 5 (read_file, write_file, ...)
Status: ready

github          [stdio] ✓ Connected
Tools: 12 (create_issue, search_code, ...)
Status: ready

jira            [http] ✗ Failed
Error: 401 Unauthorized
Last attempt: 2 minutes ago

slack           [stdio] △ Disconnected
Reason: Process exited (code 1)
```

```
# 2. 특정 서버 도구 목록
```

```
> /mcp tools github
- create_issue(owner, repo, title, body)
- create_pull_request(...)
- merge_pull_request(...)
- ...
```

```
# 3. 특정 서버 재연결
> /mcp reconnect jira
```

```
# 4. 디버깅 정보
```

```
> /mcp logs github
# 최근 stderr 출력 표시
```

List

서버 목록

Tools  
도구 확인

Reconnect  
재연결

Logs  
디버깅

# MCP 인증 패턴

## 안전한 자격증명 관리

### PATTERN 1

#### 환경변수

env 필드에 \${TOKEN} 참조. 토큰은 셸 환경에서 export. 가장 기본.

### PATTERN 2

#### Secret 관리 도구

AWS Secrets Manager, Vault에서 시작 시 export. 자동 회전 가능.

### PATTERN 3

#### OAuth 흐름

/mcp login으로 브라우저 인증. 토큰 자동 갱신. 사용자 친화.

### PATTERN 4

#### mTLS / 인증서

클라이언트 인증서로 인증. 사내 PKI 통합. 최고 보안.

### BEST PRACTICE

토큰을 settings.json에 직접 넣지 마세요. 환경변수 참조를 사용하고 실제 토큰은 사내 secret 관리 도구에서 가져오는 패턴이 표준입니다.

Env var  
기본

Secrets Mgr  
권장

OAuth  
편의

mTLS  
최고 보안

# MCP 서버 디스커버리

## 어디서 새 서버를 찾는가

| Source      | URL                                                                                                   | 특징             |
|-------------|-------------------------------------------------------------------------------------------------------|----------------|
| 공식 저장소      | <a href="https://github.com/modelcontextprotocol/servers">github.com/modelcontextprotocol/servers</a> | 검증된 서버 (40+)   |
| Awesome MCP | <a href="https://github.com/punkpeye/awesome-mcp-servers">github.com/punkpeye/awesome-mcp-servers</a> | 커뮤니티 모음 (300+) |
| mcp.so      | <a href="https://mcp.so">mcp.so</a>                                                                   | 서버 디렉토리        |
| 사내 저장소      | 사내 GitLab/Nexus                                                                                       | 검증된 사내 서버      |

### SECURITY WARNING

커뮤니티 MCP 서버는 사용 전 코드 검토 필수입니다. 신뢰되지 않은 서버는 자격증명을 탈취하거나 악성 행동을 할 수 있습니다.

Official  
검증

Community  
풍부

Internal  
사내

Review  
필수

# MCP 보안 고려사항

## 안전한 통합 패턴

### RISK 1

#### 악성 서버

검증 안 된 MCP 서버가 자격증명 탈취 또는 데이터 유출 가능

### RISK 2

#### 과도한 권한

MCP가 사용하는 토큰이 너무 광범위. 최소 권한 원칙 적용 필요

### RISK 3

#### 입력 검증 부재

MCP 응답이 메인 컨텍스트에 그대로 들어감. prompt injection 가능

### RISK 4

#### 의존성 위험

npm으로 받는 패키지가 변조될 수 있음. 사내 미러 사용 권장

### MITIGATIONS

managed-settings의 mcpPolicy allowlist로 허용 서버만 사용, 사내 npm 미러 강제, MCP 토큰은 최소 권한 + 짧은 만료, prompt injection 방어 패턴 적용.

#### Allowlist

서버 통제

#### Least-priv

최소 권한

#### Mirror

npm 미러

#### Injection

주입 방어

# MCP 성능 최적화

## 응답 속도와 자원 관리

### TIP 1

#### 필요한 서버만

안 쓰는 MCP는 settings에서 제거. 시작 시간과 메모리 절약.

### TIP 2

#### stdio 우선

같은 머신이면 stdio가 가장 빠름. http/sse는 네트워크 지연 있음.

### TIP 3

#### 결과 캐싱

list\_tables나 describe 같은 정적 정보는 클라이언트 측 캐싱 가능.

### TIP 4

#### Pagination

큰 결과는 페이지 단위로. 한번에 전체 가져오면 토큰 폭증.

## MEASURE

OpenTelemetry로 각 MCP 호출의 응답 시간을 측정하세요. 느린 서버를 식별하고 최적화하거나 다른 방식으로 대체할 수 있습니다.

**Minimal**  
필수만

**stdio**  
빠름

**Cache**  
재사용

**Paginate**  
분할

# MCP 트러블슈팅

## 자주 발생하는 문제

| 증상                | 원인               | 해결            |
|-------------------|------------------|---------------|
| Connection failed | 서버 시작 실패, env 없음 | /mcp logs로 확인 |
| 401 Unauthorized  | 토큰 만료 또는 부정확     | 환경변수 재설정      |
| Tool not found    | 서버 버전 불일치        | npm 패키지 업데이트  |
| Timeout           | 느린 응답, 네트워크      | timeout 늘림    |
| Disconnect 반복     | 서버 크래시, 메모리      | 로그로 원인 추적     |
| Permission denied | OS 권한 부족         | 경로 권한 확인      |

/mcp logs

진단 명령

env

환경변수

npm update

버전

timeout

지연



# 멀티 서버 오케스트레이션

## 여러 MCP를 함께 활용

orchestration

```
# 실전 시나리오: GitHub + JIRA + Slack 통합 워크플로

# 사용자 요청
> 이번 주 머지된 PR을 분석해서 관련 JIRA 티켓 상태를 업데이트하고
  완료 사항을 #engineering 채널에 요약 게시해줘

# Claude의 동작 (멀티 MCP 자동 조율)
1. GitHub MCP
  → search_pull_requests(query: 'is:merged merged:>=2026-05-19')
  → 결과: 23개 PR

2. 각 PR에서 JIRA 티켓 키 추출 (커밋 메시지의 PROJ-123)

3. JIRA MCP
  → 각 티켓 get_issue로 상태 확인
  → transition_issue로 'Done' 상태로 변경
  → add_comment로 머지된 PR 링크 추가

4. Slack MCP
  → send_message(channel: '#engineering', text: '...')
  → 메시지 내용: 머지된 PR 수, 관련 티켓 목록, 주요 변경사항

# 결과
- 23개 PR 자동 분석
- 18개 JIRA 티켓 Done 처리
- Slack에 깔끔한 요약 게시
- 사람 개입 없이 5분 내 완료
```

3 servers

동시 활용

Workflow

워크플로

Automated

자동화

5 min

완료 시간

# Part 5 Recap

## MCP 기본과 공식 서버 핵심 정리

### TAKEAWAY 1

#### MCP 프로토콜

Tools/Resources/Prompts 3대 primitive, JSON-RPC 기반 표준

### TAKEAWAY 2

#### 3가지 통신

stdio (로컬 빠름), http (원격 중앙), sse (스트리밍)

### TAKEAWAY 3

#### 5대 공식 서버

filesystem, github, slack, postgres, jira 즉시 활용 가능

### TAKEAWAY 4

#### 보안 + 성능

allowlist + 최소 권한 + 사내 미러 + 캐싱 + 페이지네이션

### NEXT - PART 6

Part 6은 사내 MCP 서버 구축입니다. TypeScript SDK로 처음부터 끝까지 사내 MCP 서버를 작성합니다.

Part 5

완료

Part 6

Custom MCP (20p)

Part 7-8

Commands + Recap (25p)

~1.5h

잔여 시간

CHAPTER 04 – PART 06

# Settings – MCP

settings.json, Hooks, MCP 심화



# Part 6 / MCP 사내 서버 구축

## TypeScript SDK로 처음부터 끝까지

### PART OBJECTIVE

사내 시스템 통합을 위한 MCP 서버를 TypeScript SDK로 직접 작성합니다.

스켈레톤, tool 등록, resource 등록, prompt 등록, 인증, 배포까지 모두 다룹니다.

#### TOPIC 1

##### SDK 셋업

TypeScript MCP SDK 설치, 기본 서버 스켈레톤, 빌드 환경

#### TOPIC 2

##### Primitive 등록

Tools, Resources, Prompts 3종을 코드로 정의하고 등록

#### TOPIC 3

##### 인증과 보안

OAuth 2.0, mTLS, 권한 검증, 입력 검증 패턴

#### TOPIC 4

##### 배포와 운영

Docker 패키징, 사내 레지스트리, 모니터링, 로깅

20 Slides

Part 6 분량

~40 min

예상 강의 시간

TypeScript

SDK 언어

Production

실무 수준

# 왜 사내 MCP 서버인가

## 공식 서버로 부족한 경우

### CASE 1

#### 사내 시스템 통합

사내 ticket 시스템, ERP,  
인사 시스템 같은 외부 비공  
개 도구 통합

### CASE 2

#### 도메인 특화 로직

금융 거래 조회, 의료 데이터  
검색 같은 도메인 전용 비즈  
니스 로직

### CASE 3

#### 데이터 정규화

여러 사내 시스템 데이터를  
통합 형식으로 노출하는 어  
댑터

### CASE 4

#### 보안 정책

권한 검사, 감사 로깅, 마스  
킹 같은 사내 보안 정책 일관  
적용

### KEY BENEFIT

사내 MCP 서버를 만들면 모든 Claude Code 사용자가 동일한 사내 시스템을 동일한 방식으로 접근합니다. 사용자별 토큰 관리, 권한 적용, 감사가 한 곳에서 통제됩니다.

#### Integration

통합

#### Domain

도메인 로직

#### Normalize

정규화

#### Governance

거버넌스

# TypeScript SDK 셋업

## 프로젝트 초기화

```
# 1. 새 프로젝트 디렉토리 생성
$ mkdir company-mcp-server
$ cd company-mcp-server
$ npm init -y
# 2. MCP SDK와 의존성 설치
$ npm install @modelcontextprotocol/sdk
$ npm install -D typescript @types/node tsx
$ npm install zod # 입력 검증용

# 3. tsconfig.json 생성
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "Node16",
    "moduleResolution": "Node16",
    "esModuleInterop": true,
    "strict": true,
    "outDir": "./dist",
    "rootDir": "./src",
    "declaration": true
  },
  "include": ["src/**/*"]
}
```

```
# 4. package.json scripts
{
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js",
    "dev": "tsx watch src/index.ts"
  },
  "bin": {
    "company-mcp": "./dist/index.js"
  }
}
```

# 5. 디렉토리 구조

```
src/
├─ index.ts      # 진입점
├─ server.ts     # MCP 서버 설정
├─ tools/        # 도구 구현
├─ resources/    # 리소스 구현
└─ auth.ts       # 인증 로직
```

setup

SDK

공식 패키지

TypeScript

타입 안전

zod

런타임 검증

tsx watch

개발 편의

# 기본 서버 스켈레톤

## index.ts 최소 구현

```
// src/index.ts
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";

const server = new Server(
  {
    name: "company-mcp-server",
    version: "1.0.0",
  },
  {
    capabilities: {
      tools: {},
      resources: {},
      prompts: {},
    },
  }
);

// 도구 등록은 다음 슬라이드에서
```

```
async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.error("Company MCP Server running on stdio");
}

main().catch((err) => {
  console.error("Server error:", err);
  process.exit(1);
});

// 실행
// $ npm run build && npm start
// 또는 개발 시: $ npm run dev
```

skeleton

Server

핵심 클래스

StdioTransport

통신 방식

capabilities

제공 기능

stderr

로그 출력

# Tools 등록 / 정의

## 도구 메타데이터와 핸들러

```
// src/tools/employee.ts
import { z } from "zod";

// 입력 스키마 (Zod로 검증)
export const getEmployeeSchema = z.object({
  employee_id: z.string().describe("사원 ID (예: emp_12345)"),
});

// 도구 메타데이터
export const getEmployeeTool = {
  name: "get_employee",
  description: "사원 ID로 사원 정보 조회",
  inputSchema: {
    type: "object",
    properties: {
      employee_id: {
        type: "string",
        description: "사원 ID (예: emp_12345)",
      },
    },
    required: ["employee_id"],
  },
};
```

```
// 도구 핸들러 (실제 로직)
export async function handleGetEmployee(args: unknown) {
  // 입력 검증
  const { employee_id } = getEmployeeSchema.parse(args);

  // 사내 API 호출 (예시)
  const response = await fetch(
    `https://hr-api.company.com/employees/${employee_id}`,
    { headers: { "Authorization": `Bearer ${process.env.HR_TOKEN}` } }
  );

  if (!response.ok) throw new Error(`HR API error: ${response.status}`);

  const employee = await response.json();
  return employee;
}
```

tool def

| zod   | name+desc | JSON Schema | async fetch |
|-------|-----------|-------------|-------------|
| 입력 검증 | 메타데이터     | 입력 정의       | 실제 로직       |



# Tools 등록 / 핸들러 연결

## server에 도구 등록

```
// src/server.ts
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import {
  ListToolsRequestSchema,
  CallToolRequestSchema,
} from "@modelcontextprotocol/sdk/types.js";
import {
  getEmployeeTool, handleGetEmployee
} from "../tools/employee.js";
import {
  searchTicketsTool, handleSearchTickets
} from "../tools/tickets.js";

export function setupServer(server: Server) {
  // 1. tools 목록 응답
  server.setRequestHandler(ListToolsRequestSchema, async () => ({
    tools: [
      getEmployeeTool,
      searchTicketsTool,
      // ... 다른 도구들
    ],
  }));
```

```
// 2. tool 호출 처리
server.setRequestHandler(CallToolRequestSchema, async (request) => {
  const { name, arguments: args } = request.params;

  switch (name) {
    case "get_employee":
      return { content: [{
        type: "text",
        text: JSON.stringify(await handleGetEmployee(args), null, 2)
      }]};
    case "search_tickets":
      return { content: [{
        type: "text",
        text: JSON.stringify(await handleSearchTickets(args), null, 2)
      }]};
    default:
      throw new Error(`Unknown tool: ${name}`);
  }
});
```

register

ListTools

목록 응답

CallTool

호출 처리

switch

디스패치

content

응답 형식

# Resources 등록

## 읽기 전용 데이터 노출

resources

```
// src/resources/wiki.ts
import {
  ListResourcesRequestSchema,
  ReadResourceRequestSchema,
} from "@modelcontextprotocol/sdk/types.js";

export function setupWikiResources(server: Server) {
  // 1. 사용 가능한 리소스 목록
  server.setRequestHandler(ListResourcesRequestSchema, async () => ({
    resources: [
      {
        uri: "wiki://onboarding",
        name: "Onboarding Guide",
        description: "신입 사원 온보딩 가이드",
        mimeType: "text/markdown",
      },
      {
        uri: "wiki://architecture",
        name: "System Architecture",
        description: "사내 시스템 아키텍처 문서",
        mimeType: "text/markdown",
      },
    ],
  }));
}
```

```
// 2. 리소스 콘텐츠 반환
server.setRequestHandler(ReadResourceRequestSchema, async (request) => {
  const { uri } = request.params;

  // 사내 wiki API 호출
  const docName = uri.replace("wiki://", "");
  const response = await fetch(
    `https://wiki.company.com/api/pages/${docName}`,
    { headers: { "Authorization": `Bearer ${process.env.WIKI_TOKEN}` } }
  );

  const markdown = await response.text();
  return {
    contents: [{
      uri,
      mimeType: "text/markdown",
      text: markdown,
    }],
  };
});
}
```

List  
목록 응답

Read  
콘텐츠 응답

URI scheme  
식별

mimeType  
타입 지정

# Prompts 등록

## 재사용 가능한 템플릿

```
// src/prompts/standard.ts
import {
  ListPromptsRequestSchema,
  GetPromptRequestSchema,
} from "@modelcontextprotocol/sdk/types.js";

export function setupPrompts(server: Server) {
  // 1. 프롬프트 목록
  server.setRequestHandler(ListPromptsRequestSchema, async () => ({
    prompts: [
      {
        name: "incident-response",
        description: "장애 대응 표준 절차",
        arguments: [
          { name: "service", description: "장애 서비스명", required: true },
          { name: "severity", description: "심각도 (P0~P4)", required: true },
        ],
      },
      {
        name: "code-review-checklist",
        description: "사내 표준 코드 리뷰 체크리스트",
        arguments: [
          { name: "pr_url", description: "PR URL", required: true },
        ],
      },
    ],
  }));
}
```

```
// 2. 프롬프트 콘텐츠
server.setRequestHandler(GetPromptRequestSchema, async (request) => {
  const { name, arguments: args } = request.params;

  if (name === "incident-response") {
    return {
      messages: [{
        role: "user",
        content: { type: "text", text:
`다음 장애를 대응해 주세요:
- 서비스: ${args?.service}
- 심각도: ${args?.severity}`
        },
      ]},
    ];
  }
  throw new Error(`Unknown prompt: ${name}`);
});
}
```

prompts

List  
템플릿 목록

Get  
템플릿 반환

arguments  
동적 값

Reusable  
재사용

# 입력 검증 패턴

## Zod로 안전한 처리

```
// src/tools/tickets.ts
import { z } from "zod";
// 강력한 스키마 정의
export const searchTicketsSchema = z.object({
  query: z.string()
    .min(3, "쿼리는 최소 3자 이상")
    .max(200, "쿼리는 최대 200자")
    .refine(
      (s) => !/<script|javascript:|eval\\(\\/i.test(s),
      "위험한 패턴 감지"
    ),
  status: z.enum(["open", "in_progress", "closed"]).optional(),
  priority: z.enum(["low", "medium", "high", "critical"]).optional(),
  limit: z.number().int().min(1).max(100).default(20),
});
export async function handleSearchTickets(args: unknown) {
  // 입력 검증 (실패 시 ZodError throw)
  const params = searchTicketsSchema.parse(args);
  // SQL injection 방지: parameterized query 사용
  const tickets = await db.query(
    `SELECT id, title, status FROM tickets
     WHERE title LIKE ? AND status = ?
     LIMIT ?`,
    [`%${params.query}%`, params.status ?? "open", params.limit]
  );
};
```

validation

```
// 출력 마스킹 (민감 정보 제거)
return tickets.map((t) => ({
  id: t.id,
  title: t.title,
  status: t.status,
  // assignee_email은 의도적으로 제외
})));
} // SQL injection 방지: parameterized query 사용
const tickets = await db.query(
  `SELECT id, title, status FROM tickets
   WHERE title LIKE ? AND status = ?
   LIMIT ?`,
  [`%${params.query}%`, params.status ?? "open", params.limit]
);

// 출력 마스킹 (민감 정보 제거)
return tickets.map((t) => ({
  id: t.id,
  title: t.title,
  status: t.status,
  // assignee_email은 의도적으로 제외
})));
}
```

Zod

런타임 검증

min/max

길이 제한

enum

값 제한

Parameterized

SQL 안전

# 서버 인증

## 클라이언트 신원 확인

```
// src/auth.ts
import jwt from "jsonwebtoken";

export interface AuthContext {
  userId: string;
  email: string;
  groups: string[];
  permissions: string[];
}

const JWT_SECRET = process.env.JWT_SECRET ?? "";

// MCP 요청에서 인증 정보 추출
export function authenticate(request: any): AuthContext {
  // stdio: 환경변수에서 토큰 (개발용)
  const token = process.env.MCP_AUTH_TOKEN ??
    request.params?.auth?.token;

  if (!token) {
    throw new Error("Authentication required");
  }
}
```

```
try {
  const decoded = jwt.verify(token, JWT_SECRET) as any;
  return {
    userId: decoded.sub,
    email: decoded.email,
    groups: decoded.groups ?? [],
    permissions: decoded.permissions ?? [],
  };
} catch (err) {
  throw new Error("Invalid or expired token");
}

// 권한 검사
export function requirePermission(ctx: AuthContext, perm: string) {
  if (!ctx.permissions.includes(perm)) {
    throw new Error(`Permission denied: ${perm}`);
  }
}

// 도구 핸들러에서 사용
export async function handleGetEmployee(args: unknown, request: any) {
  const ctx = authenticate(request);
  requirePermission(ctx, "hr:read");
  // ... 실제 로직
}
```

auth

JWT

토큰 검증

Context

사용자 정보

Permission

권한 검사

Per-tool

도구별

# 에러 처리 패턴

## MCP 표준 에러 응답

```
// src/errors.ts
import { ErrorCode, McpError } from "@modelcontextprotocol/sdk/types.js";

// 표준 MCP 에러
export class AuthError extends McpError {
  constructor(message: string) {
    super(ErrorCode.InvalidRequest, message);
  }
}

export class PermissionError extends McpError {
  constructor(perm: string) {
    super(ErrorCode.InvalidRequest, `Permission denied: ${perm}`);
  }
}

export class NotFoundError extends McpError {
  constructor(resource: string) {
    super(ErrorCode.InvalidParams, `Not found: ${resource}`);
  }
}
```

```
// 도구 핸들러에서 사용
export async function handleGetEmployee(args: unknown, request: any) {
  try {
    const ctx = authenticate(request);
    requirePermission(ctx, "hr:read");
    const { employee_id } = getEmployeeSchema.parse(args);

    const response = await fetch(`...${employee_id}`);
    if (response.status === 404) {
      throw new NotFoundError(`Employee: ${employee_id}`);
    }
    if (response.status === 401) {
      throw new AuthError("HR API authentication failed");
    }

    return await response.json();
  } catch (err) {
    if (err instanceof McpError) throw err; // re-throw

    // 예상 못 한 에러는 로그만
    console.error("Unexpected:", err);
    throw new McpError(
      ErrorCode.InternalError,
      "Internal server error"
    );
  }
}
```

errors

McpError

표준 클래스

ErrorCode

에러 분류

try/catch

래핑

Re-throw

재던지기

# 로깅과 관측 가능성

## 운영을 위한 추적

```
// src/logger.ts
import pino from "pino";

export const logger = pino({
  level: process.env.LOG_LEVEL || "info",
  formatters: {
    level: (label) => ({ level: label }),
  },
  base: {
    service: "company-mcp-server",
    version: process.env.npm_package_version,
  },
});
```

```
// 도구 핸들러에서 구조화 로깅
export async function handleSearchTickets(args: unknown, req: any) {
  const startTime = Date.now();
  const ctx = authenticate(req);

  logger.info({
    user_id: ctx.userId,
    tool: "search_tickets",
    args: { ...args, sensitive_removed: true },
  }, "Tool invocation");

  try {
    const result = await /* 실제 로직 */;

    logger.info({
      user_id: ctx.userId,
      tool: "search_tickets",
      duration_ms: Date.now() - startTime,
      result_count: result.length,
    }, "Tool success");

    return result;
  } catch (err) {
    logger.error({
      user_id: ctx.userId,
      tool: "search_tickets",
      error: err instanceof Error ? err.message : String(err),
      duration_ms: Date.now() - startTime,
    }, "Tool failed");
    throw err;
  }
}
```

logging

pino

로그 라이브러리

Structured

JSON 형식

Per-call

호출별

Duration

시간 측정

# 서버 테스트

## 단위와 통합 테스트

tests

```
// tests/employee.test.ts
import { describe, it, expect, vi } from "vitest";
import {
  handleGetEmployee, getEmployeeSchema
} from "../src/tools/employee";

describe("getEmployeeSchema", () => {
  it("유효한 employee_id 통과", () => {
    expect(() => getEmployeeSchema.parse({
      employee_id: "emp_12345"
    })).not.toThrow();
  });

  it("빈 employee_id 거부", () => {
    expect(() => getEmployeeSchema.parse({
      employee_id: ""
    })).toThrow();
  });
});
```

```
describe("handleGetEmployee", () => {
  it("정상 조회 시 employee 객체 반환", async () => {
    // fetch mock
    global.fetch = vi.fn().mockResolvedValue({
      ok: true,
      json: async () => ({ id: "emp_1", name: "Alice" }),
    });

    const result = await handleGetEmployee(
      { employee_id: "emp_1" },
      { headers: { authorization: "Bearer test" } }
    );

    expect(result.name).toBe("Alice");
  });
});

// 실행
// $ npm test
```

**vitest**

테스트 도구

**Schema test**

스키마 검증

**Handler test**

핸들러 검증

**Mock fetch**

외부 차단



# Docker 패키징

## 컨테이너로 배포

### Dockerfile

```
# Dockerfile
FROM node:20-alpine AS builder
WORKDIR /app

# 의존성 설치 (캐시 최적화)
COPY package*.json ./
RUN npm ci --only=production

# 빌드
COPY tsconfig.json ./
COPY src ./src
RUN npm install -D typescript && npx tsc

# 런타임 이미지
FROM node:20-alpine
WORKDIR /app

# 비root 사용자
RUN addgroup -S mcp && adduser -S mcp -G mcp

COPY --from=builder --chown=mcp:mcp /app/node_modules ./node_modules
COPY --from=builder --chown=mcp:mcp /app/dist ./dist
COPY --from=builder --chown=mcp:mcp /app/package.json ./

USER mcp

# 헬스체크
HEALTHCHECK --interval=30s --timeout=3s \
  CMD node -e "process.exit(0)" || exit 1

# 메타데이터
LABEL maintainer="platform@company.com"
LABEL version="1.0.0"

ENTRYPOINT ["node", "dist/index.js"]

# 빌드
# $ docker build -t registry.company.com/mcp/company-mcp:1.0.0 .
# 푸시
# $ docker push registry.company.com/mcp/company-mcp:1.0.0
```

Multi-stage

빌드 분리

Non-root

보안

Alpine

경량

Healthcheck

운영

# HTTP 서버 모드

## 원격 MCP 서버로 배포

http server

```
// src/http-server.ts - HTTP 모드 진입점
import express from "express";
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import {
  SSEServerTransport
} from "@modelcontextprotocol/sdk/server/sse.js";
import { setupServer } from "../server.js";

const app = express();
app.use(express.json());

const mcpServer = new Server({}, {});
setupServer(mcpServer);

// SSE endpoint
app.get("/sse", async (req, res) => {
  // 인증
  const auth = req.headers.authorization;
  if (!auth?.startsWith("Bearer ")) {
    return res.status(401).json({ error: "Unauthorized" });
  }

  const transport = new SSEServerTransport("/messages", res);
  await mcpServer.connect(transport);
});
```

```
// Messages endpoint
app.post("/messages", express.raw({ type: "application/json" })),
  async (req, res) => {
    // 처리...
  }
);

// Health check
app.get("/health", (req, res) => res.json({ status: "ok" }));

const port = process.env.PORT || 8080;
app.listen(port, () => {
  console.error(`MCP HTTP server on :${port}`);
});
```

Express

HTTP 프레임워크

SSE transport

실시간

Bearer

인증

/health

모니터링

# Kubernetes 배포

## 사내 클러스터에 운영

```
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: company-mcp-server
  namespace: platform
spec:
  replicas: 3
  selector:
    matchLabels: { app: company-mcp }
  template:
    metadata:
      labels: { app: company-mcp }
```

```
spec:
  containers:
    - name: mcp
      image: registry.company.com/mcp/company-mcp:1.0.0
      ports: [{ containerPort: 8080 }]
      env:
        - name: JWT_SECRET
          valueFrom: { secretKeyRef: { name: mcp-secrets, key: jwt }}
        - name: HR_TOKEN
          valueFrom: { secretKeyRef: { name: mcp-secrets, key: hr }}
      resources:
        requests: { memory: "128Mi", cpu: "100m" }
        limits: { memory: "512Mi", cpu: "500m" }
      livenessProbe:
        httpGet: { path: /health, port: 8080 }
        periodSeconds: 30
    ---
  apiVersion: v1
  kind: Service
  metadata:
    name: company-mcp
    namespace: platform
  spec:
    selector: { app: company-mcp }
    ports:
      - port: 443
        targetPort: 8080
        type: ClusterIP
    ---
  # Ingress로 외부 노출
  # 사용자: https://mcp.company.com/sse 접근
```

k8s yaml

3 replicas

HA

Secrets

인증 분리

Resources

자원 제한

Liveness

헬스체크

# 실전 예시 / 회사 위키 검색

## 완성된 사내 MCP

real example

```
# settings.json (사용자 측)
{
  "mcpServers": {
    "company-wiki": {
      "type": "sse",
      "url": "https://mcp.company.com/sse",
      "headers": {
        "Authorization": "Bearer ${COMPANY_MCP_TOKEN}"
      }
    }
  }
}

# 사용 예시
> 우리 회사의 코딩 컨벤션 문서를 찾아서 요약해줘
```

```
# Claude의 동작
1. company-wiki MCP의 search_documents 도구 호출
   → query: "coding convention"
2. 결과 받음:
   - "engineering/coding-style.md"
   - "frontend/react-guidelines.md"
   - "backend/python-pep8-strict.md"
3. 각 문서를 read_document로 가져옴
4. 요약 응답:
   "우리 회사 코딩 컨벤션은 다음과 같습니다:
   - JavaScript: ESLint Airbnb config 사용
   - Python: PEP 8 + 사내 추가 규칙 5가지
   - React: Hooks 우선, class component 금지
   ..."
```

```
# 운영 메트릭 (3개월 운영 후)
- 일 평균 호출: 4,200건
- p99 응답 시간: 180ms
- 사용자: 230명
- 만족도: 4.6 / 5.0
```

Production

실 운영

4200/day

일 호출

180ms p99

응답 시간

4.6/5

만족도

# MCP 서버 Best Practices

## 운영 가능한 품질

### BP 1

#### 입력 검증

Zod 같은 라이브러리로  
strict한 스키마 정의, 모든  
입력 검증

### BP 2

#### 권한 분리

도구별 명시적 권한 요구,  
AuthContext로 사용자 신  
원 추적

### BP 3

#### 에러 명확화

McpError로 표준화, 내부  
상세는 노출 금지, 사용자 친  
화 메시지

### BP 4

#### 구조화 로깅

JSON 로그, user\_id 포함,  
duration 측정, 민감 정보  
제거

### BP 5 / 점진 도입

MCP 서버를 한 번에 다 만들지 마세요. 가장 자주 쓰는 작업 1-2개로 시작하고 사용자 피드백을 받으며 확장하는 것이 성공 패턴입니다.

#### Validation

입력

#### AuthZ

권한

#### Errors

에러

#### Logging

관측

# 버전 관리와 업데이트

## 하위 호환성 유지

versioning

```
# Semantic Versioning (SemVer)
# MAJOR.MINOR.PATCH
# - MAJOR: 호환 불가능한 변경 (도구 제거, 시그니처 변경)
# - MINOR: 호환 가능한 기능 추가 (새 도구)
# - PATCH: 버그 수정, 내부 개선

# 호환 가능한 변경 (MINOR/PATCH)
# 새 도구 추가
- get_employee
+ get_employee
+ list_employees    # 새 도구 (사용자 기존 코드 영향 없음)

# 도구에 optional 인자 추가
get_employee(employee_id)
get_employee(employee_id, include_history=false) # default 있으면 OK

# 호환 불가능 변경 (MAJOR)
# 도구 시그니처 변경 - 이전 사용자 깨짐
get_employee(employee_id) → get_employee_v2(emp_uuid)
```

```
# 변경 전략
1. 새 도구를 _v2 suffix로 추가 (기존 유지)
2. 일정 기간 두 버전 모두 제공 (deprecation 안내)
3. 사용자 마이그레이션 완료 후 구버전 제거 (다음 major)

# Changelog 유지
## [1.2.0] - 2026-05-25
### Added
- list_employees 도구 추가
### Changed
- get_employee 응답에 department 필드 추가
```

SemVer

버전 규칙

\_v2 suffix

Breaking 회피

Deprecation

안내 기간

Changelog

변경 기록

# Part 6 Recap

## MCP 사내 서버 구축 핵심 정리

### TAKEAWAY 1

#### SDK 구조

TypeScript SDK + Zod 검증  
+ pino 로깅, 표준 구조 정착

### TAKEAWAY 2

#### 3 Primitive

Tools 등록, Resources 노출  
, Prompts 템플릿 모두 구현

### TAKEAWAY 3

#### 보안 다층

입력 검증 + 인증 + 권한 +  
마스킹 + 감사 로깅

### TAKEAWAY 4

#### 운영 자동화

Docker + Kubernetes +  
모니터링 + 점진 배포

### NEXT - PART 7

Part 7은 Custom Slash Commands입니다. `.claude/commands` 디렉토리, 명령 파일 형식, 인자 처리, 팀 자산화를 15장에 걸쳐 다룹니다.

### Part 6

완료

### Part 7

Slash Commands (15p)

### Part 8-9

Recap + Labs (20p)

### ~1h

잔여 시간

CHAPTER 04 – PART 07

# Settings – Custom Slash Commands

settings.json, Hooks, MCP 심화





# Part 7 / Custom Slash Commands

## 팀 워크플로 자동화

### PART OBJECTIVE

Markdown 파일로 사용자 정의 슬래시 명령을 만들고 팀 자산으로 공유합니다. 인자 처리, frontmatter, 권한 통제까지 모두 다룹니다.

#### TOPIC 1

##### 기본 구조

.claude/commands 디렉토리, 명령 파일 형식, 호출 방법

#### TOPIC 2

##### 인자와 변수

\$ARGUMENTS 변수, frontmatter 설정, 동적 콘텐츠 생성

#### TOPIC 3

##### 팀 자산화

Git으로 공유, 명령 라이브러리, 발견과 사용

#### TOPIC 4

##### 실전 패턴

리뷰, 배포, 디버깅, 보고서 같은 실제 활용 예시

15 Slides

Part 7 분량

~30 min

예상 강의 시간

Markdown

간단함

Team asset

공유 자산

# Slash Command 기본

## 명령 파일 생성과 호출

basics

# 1. 디렉토리 생성 (프로젝트 또는 사용자 전역)

```
$ mkdir -p .claude/commands
```

# 또는 사용자 전역:

```
$ mkdir -p ~/.claude/commands
```

# 2. 명령 파일 생성 (Markdown)

```
$ cat > .claude/commands/review.md <<'EOF'
```

다음 git diff를 코드 리뷰해 주세요.

관점: 가독성, 안전성, 성능.

출력 형식: 우선순위별 정리 (Critical/Important/Suggestion).

```
EOF
```

# 3. claude에서 호출

```
> /review
```

# 또는 인자와 함께:

```
> /review HEAD~3
```

# 4. 파일 이름이 명령 이름이 됨

```
# review.md → /review
```

```
# deploy.md → /deploy
```

```
# fix-bug.md → /fix-bug
```

# 5. 자동 노출

```
> /
```

# 슬래시 입력 시 사용 가능한 명령 목록 표시

```
/review
```

```
/deploy
```

```
/fix-bug
```

```
...
```

# 6. 우선순위

# 프로젝트 명령 > 사용자 명령 > 빌트인 명령

# 같은 이름이면 더 상위가 사용됨

**.md file**

명령 파일

**filename**

명령 이름

**/cmd**

호출 방식

**Auto-list**

자동 노출

# 인자가 있는 명령

## \$ARGUMENTS 변수

arguments

```
# .claude/commands/fix-issue.md
다음 GitHub Issue 번호의 문제를 수정해 주세요: # $ARGUMENTS
```

작업 절차:

1. gh issue view \$ARGUMENTS 로 issue 내용 확인
2. 관련 파일을 Grep으로 탐색
3. 문제 원인 파악
4. 수정 사항 적용
5. 테스트 작성 또는 갱신
6. PR 생성 시 "Closes # \$ARGUMENTS" 명시

# 호출 예시

```
> /fix-issue 1234
# $ARGUMENTS가 "1234"로 치환됨
```

# 결과적으로 Claude가 받는 프롬프트

```
"다음 GitHub Issue 번호의 문제를 수정해 주세요: #1234
작업 절차: 1. gh issue view 1234 로 issue 내용 확인 ..."
```

# 여러 인자

```
> /fix-issue 1234 high
# $ARGUMENTS = "1234 high" (전체가 한 문자열로)
```

# 더 정교한 처리는 명령 내부에서 prompting

"인자가 두 부분이면 첫째는 issue 번호,  
둘째는 우선순위로 처리해 주세요."

# 빈 호출

```
> /fix-issue
# $ARGUMENTS = "" (빈 문자열)
# 명령 내부에서 처리 방법 안내
```

\$ARGUMENTS

인자 변수

Single string

한 문자열

Parsing

명령 내부

Optional

없어도 OK

# Frontmatter 옵션

## 명령 메타데이터

```
# .claude/commands/security-review.md
---
description: 보안 관점 코드 리뷰 (취약점, secret, OWASP Top 10)
allowed-tools:
  - Read
  - Grep
  - Glob
model: claude-opus-4-5      # 보안 분석은 Opus 권장
disable-model-invocation: false
---
```

다음 git diff를 보안 관점에서 검토해 주세요: \$ARGUMENTS

검토 항목:

1. OWASP Top 10 패턴 검사
2. 시크릿 또는 자격증명 노출 여부
3. SQL injection 가능성
4. XSS 취약점
5. 권한 우회 가능성

```
# Frontmatter 필드 설명
description: /commands 목록에 표시되는 설명
allowed-tools: 이 명령 실행 중 허용되는 도구 (보안 강화)
model: 이 명령에 강제로 사용할 모델
disable-model-invocation: true면 사용자 모델 선택 비활성
argument-hint: 인자 형식 힌트 (자동완성에 도움)
```

# 사용 예시 (목록 표시 시)

```
> /
  /review      - PR 자동 리뷰
  /security-review - 보안 관점 코드 리뷰 (취약점, ...)
  /deploy      - 표준 배포 절차
```

frontmatter

description

명령 설명

allowed-tools

권한 줌

model

모델 강제

hint

인자 힌트

# 명령별 도구 제한

## 명령 단위 권한 통제

```
# .claude/commands/read-only-analysis.md
---
description: 읽기 전용 분석 (코드 수정 절대 안 함)
allowed-tools:
  - Read
  - Grep
  - Glob
  - WebFetch
disallowed-tools:
  - Edit
  - Write
  - MultiEdit
  - Bash
---
```

이 명령은 분석만 수행하고 절대 수정하지 않습니다.

\$ARGUMENTS 에 대해 다음 관점에서 분석해 주세요:

- 코드 구조와 의도
- 잠재적 문제점
- 개선 제안 (실행은 안 함)

```
# .claude/commands/auto-deploy.md
---
description: 자동 배포 (위험 작업, 신중히 사용)
allowed-tools:
  - Read
  - Bash(git:*)
  - Bash(npm run deploy:*)
disallowed-tools:
  - Bash(rm:*)
  - Bash(sudo:*)
---
```

\$ARGUMENTS 환경에 배포합니다.

절차:

1. git status로 깨끗한지 확인
2. npm run test 통과 확인
3. npm run deploy:\$ARGUMENTS 실행

# 효과

- 명령 단위 권한 통제로 안전성 향상
- 위험한 명령은 매우 좁은 권한만

tool restrict

allowed

허용 도구

disallowed

차단 도구

Per-cmd

명령별

Safe

안전

# 실전 명령 1 / PR 리뷰

## /review-pr

```
# .claude/commands/review-pr.md
```

```
---
```

```
description: PR 자동 종합 리뷰 (코드 + 보안 + 테스트)
```

```
allowed-tools:
```

- Read
- Grep
- Glob
- Bash(gh pr:\*)
- Bash(git diff:\*)

```
model: claude-sonnet-4-5
```

```
---
```

```
PR #$ARGUMENTS 를 종합 리뷰해 주세요.
```

```
# 절차
```

1. gh pr view \$ARGUMENTS 로 PR 정보 가져오기
2. gh pr diff \$ARGUMENTS 로 변경 내용 분석
3. 다음 4가지 관점에서 검토:

```
## 1. 코드 품질
```

- 가독성, 명명, 구조
- 중복 코드, 복잡도
- SOLID 원칙 위반

```
## 2. 보안
```

- OWASP Top 10
- 시크릿 노출
- 입력 검증 부재

```
## 3. 테스트
```

- 커버리지 (변경 부분에 새 테스트가 있는가)
- 엡지 케이스 누락
- 통합 테스트 필요성

```
## 4. 운영
```

- 성능 영향
- 의존성 변경
- 마이그레이션 필요성

```
# 출력 형식
```

```
각 섹션을 분리하고, 우선순위 (Critical / Important / Suggestion)  
로 정리. 직접적인 수정 제안 코드 블록 포함.
```

review-pr

gh integration

GitHub CLI

4 perspectives

4관점

Structured

구조화 출력

Actionable

실행 가능

# 실전 명령 2 / 배포

## /deploy

deploy

```
# .claude/commands/deploy.md
---
description: 표준 배포 절차 (dev/staging/prod)
argument-hint: <env>
allowed-tools:
  - Read
  - Bash(git status)
  - Bash(git log:*)
  - Bash(npm test)
  - Bash(npm run build)
  - Bash(npm run deploy:*)
  - Bash(curl https://hooks.slack.com/*)
model: claude-sonnet-4-5
---
$ARGUMENTS 환경에 배포를 진행해 주세요.
# 사전 점검
1. git status가 깨끗한가? (dirty면 중단)
2. 마지막 커밋이 main에 있는가?
3. 다음 명령 모두 통과 확인:
  - npm test
  - npm run build
  - npm run lint
```

```
# 배포 절차
1. 배포 전 Slack #deploys 채널에 시작 알림
  curl -X POST $SLACK_HOOK \
    -d '{"text": "🚧 Deploying to $ARGUMENTS"}'

2. npm run deploy:$ARGUMENTS 실행

3. 배포 완료 후 health check
  curl https://$ARGUMENTS.company.com/health
  응답이 200 OK가 아니면 즉시 rollback 안내

4. Slack에 완료 알림
  curl -X POST $SLACK_HOOK \
    -d '{"text": "✅ Deployed to $ARGUMENTS"}'

5. 최종 보고: 배포된 commit hash, 환경, 시간

# 주의
- prod 배포는 사용자 확인 필수
```

Pre-check

사전 점검

Notify

Slack 알림

Health

검증

Audit

보고

# 실전 명령 3 / 디버깅

## /debug-error

debug-error

```
# .claude/commands/debug-error.md
---
description: 에러 로그 분석과 원인 추적
argument-hint: <error message or file path>
allowed-tools:
  - Read
  - Grep
  - Glob
  - Bash(git log:*)
  - Bash(git blame:*)
---
```

다음 에러를 분석하고 원인을 추적해 주세요:

\$ARGUMENTS

```
# 분석 절차
1. 에러 메시지에서 키워드 추출 (함수명, 파일명, 라인)
2. 관련 코드 위치 탐색
  - Grep으로 함수명 검색
  - 호출 체인 추적 (caller 찾기)
3. 최근 변경 이력 확인
  - git log --all -p --grep="<관련 키워드>"
  - git blame 으로 해당 라인 변경자와 시점
4. 가능성 있는 원인 3가지 가설 제시
  - 입력 검증 부재
  - race condition
  - null/undefined 처리 누락
  - 외부 의존성 변경
  - 기타
5. 각 가설별 검증 방법 제시
  - 어떤 로그를 추가하면 확인 가능한가
  - 어떤 테스트 케이스가 재현하는가
6. 즉시 적용 가능한 수정 제안
  - 핫픽스 (임시)
  - 근본 수정 (장기)
# 호출 예시
> /debug-error "TypeError: Cannot read property 'id' of undefined at user.js:42"
```

Trace

추적

git blame

이력

Hypotheses

가설

Verify

검증법



# 실전 명령 4 / 스프린트 요약

## /sprint-summary

sprint summary

```
# .claude/commands/sprint-summary.md
---
description: 스프린트 활동 요약 (PR, Issue, 결과)
argument-hint: <since-date>
allowed-tools:
  - Bash(gh pr:*)
  - Bash(gh issue:*)
  - Bash(git log:*)
model: claude-sonnet-4-5
---
```

\$ARGUMENTS 이후의 스프린트 활동을 요약해 주세요.

# 데이터 수집

1. gh pr list --state merged --search "merged:>=\$ARGUMENTS"
2. gh issue list --state closed --search "closed:>=\$ARGUMENTS"
3. git log --since="\$ARGUMENTS" --pretty=format:"%h %an %s"

# 분석

1. 머지된 PR을 카테고리별 분류 (feat, fix, docs, refactor, etc)
2. 가장 active한 contributor top 5
3. 가장 큰 변경 (LoC 기준) top 3
4. 클로즈된 issue를 우선순위에 따라 분류
5. 미해결 issue 중 critical/high 목록

```
# 출력 형식 (Markdown)
# Sprint Summary: $ARGUMENTS ~ now
```

## Highlights

- 머지된 PR: <count>개
- 클로즈된 Issue: <count>개
- 활동 contributors: <count>명

## PR 카테고리

- feat: <n> (주요: ...)
- fix: <n> (주요: ...)
- ...

## Top Contributors

1. @user1 (X PRs, Y commits)
- ...

## 다음 스프린트 준비

- 미해결 critical: <list>
- carry-over 후보: <list>

# 호출 예시

> /sprint-summary 2026-05-12

Aggregate

데이터 수집

Categorize

분류

Markdown

구조 출력

Repeatable

주기 사용

# 실전 명령 5 / 번역

## /translate-doc

translate

```
# .claude/commands/translate-doc.md
---
description: 문서를 한국어/영어로 번역 (사내 용어 사전 적용)
argument-hint: <file-path> <target-lang>
allowed-tools:
  - Read
  - Write
  - Edit
model: claude-sonnet-4-5
---
```

\$ARGUMENTS 의 첫 인자는 번역할 파일, 둘째는 대상 언어 (ko 또는 en) 입니다.

```
# 절차
1. 파일을 Read로 읽음
2. 사내 용어 사전 적용 (~/.claude/glossary.md 참고)
3. 자연스러운 톤으로 번역 (직역 금지)
4. 코드 블록과 명령어는 번역하지 않음
5. 결과를 같은 경로의 .ko 또는 .en suffix로 저장
```

```
# 사내 용어 사전 예시 (~/.claude/glossary.md)
- user → 사용자 (회원, 유저 사용 금지)
- deploy → 배포 (디플로이 금지)
- repository → 저장소
- pull request → PR (한글: 풀 리퀘스트 사용 금지)
- bug → 버그 (오류 X)
```

```
# 출력
1. 번역된 파일 저장 보고
2. 변경된 핵심 용어 5-10개 요약
3. 잘 번역되지 않은 부분 (있다면) 알림
```

```
# 호출 예시
> /translate-doc docs/architecture.md ko
> /translate-doc docs/architecture.ko.md en
```

Glossary

용어 사전

Natural

자연스러움

Code-aware

코드 보존

Bilingual

양방향

# 팀 명령 라이브러리

## Git으로 공유

# 1. 팀 표준 명령 저장소 구조

```
.claude/commands/
├─ review-pr.md      # PR 리뷰
├─ deploy.md         # 배포
├─ debug-error.md    # 디버깅
├─ sprint-summary.md # 스프린트 요약
├─ translate-doc.md  # 번역
├─ new-feature.md    # 기능 추가 워크플로
├─ postmortem.md     # 장애 회고
├─ security-audit.md # 보안 감사
└─ README.md         # 명령 가이드
```

# 2. Git에 커밋

```
$ git add .claude/commands/
$ git commit -m "feat: standard team commands"
$ git push
```

# 3. 팀원은 clone 후 자동 사용 가능

```
$ git clone git@github.com:company/repo.git
$ cd repo
$ claude
> / # 사용 가능한 명령 목록 자동 표시
```

# 4. 명령 가이드 README.md 권장

# Team Slash Commands

이 저장소는 사내 표준 슬래시 명령을 포함합니다.

## 사용 가능한 명령

```
- /review-pr <num>      - PR 종합 리뷰
- /deploy <env>         - 표준 배포 절차
- /debug-error <msg>    - 에러 분석
- /sprint-summary <date> - 스프린트 요약
- /translate-doc <file> <lang> - 문서 번역
```

## 새 명령 추가

.claude/commands/<name>.md 형식으로 작성하고 PR 제출

team library

Git  
공유 매체

Auto-discover  
clone 후 즉시

README  
가이드

PR-based  
추가 절차

# Slash Command vs Subagent

## 언제 어느 것을 쓰는가

### SLASH COMMAND

#### 슬래시 명령

1. 메인 세션에서 실행
2. 짧은 워크플로 자동화
3. 표준화된 사용자 입력 패턴
4. 인자 처리 단순
5. .md 파일, 빠른 작성
6. 사용 빈도 높음

### SUBAGENT

#### 서브 에이전트

1. 별도 컨텍스트에서 실행
2. 자기 완결적 큰 작업
3. 메인 컨텍스트 절약
4. 시스템 프롬프트 정교
5. 권한 분리 가능
6. 복잡한 분석 작업

Quick  
Command

Deep  
Subagent

Workflow  
Command 장점

Isolation  
Subagent 장점

# Command Best Practices

## 품질과 일관성

### BP 1

#### 명확한 description

한 줄로 핵심 기능과 인자 형식 표현.  
사용자가 빠르게 발견.

### BP 2

#### 도구 제한

allowed-tools로 필요한 도구만 명시.  
의도하지 않은 동작 차단.

### BP 3

#### 단계별 구조

본문을 절차로 구조화.  
Claude가 일관된 흐름으로 작업.

### BP 4

#### 출력 형식 명시

Markdown, JSON 등 출력 형식을 명령에서 명시해 일관성 확보.

### BP 5 / 명령 라이브러리

한 번에 모든 명령을 만들지 마세요. 팀이 매주 반복하는 작업 1-2개부터 명령화하고 실사용을 통해 확장하는 방식이 정착됩니다.

Clear

명확함

Scoped

권한

Structured

구조

Iterative

점진 도입

# 명령 발견과 학습

## 팀에 명령 도입

STEP 01

명령  
작성

STEP 02

README  
업데이트

STEP 03

Git  
커밋

STEP 04

팀 공지  
+ 데모

STEP 05

사용 피드백  
+ 개선

| 활동        | 담당        | 주기          |
|-----------|-----------|-------------|
| 새 명령 작성   | 제안자 + 리뷰어 | 필요 시        |
| PR 리뷰     | 팀 leader  | 필요 시        |
| 데모 발표     | 작성자       | 주간 stand-up |
| 사용량 분석    | DevOps    | 월간          |
| 오래된 명령 제거 | 팀 합의      | 분기별         |

Process

명령 도입

Demo

학습 매체

Metrics

사용량

Prune

정리

# Part 7 Recap

## Custom Slash Commands 핵심 정리

### TAKEAWAY 1

#### 기본 구조

.md 파일 + frontmatter,  
파일 이름이 명령 이름

### TAKEAWAY 2

#### \$ARGUMENTS

인자를 단순 치환, 명령 내부  
prompting으로 정교한 처리

### TAKEAWAY 3

#### Frontmatter

allowed-tools로 권한 좁힘  
, model 강제, description  
으로 발견 가능

### TAKEAWAY 4

#### 팀 자산화

Git으로 공유, README 가이드,  
점진 도입, 주기 정리

### NEXT - PART 8

Part 8은 통합 패턴과 Troubleshooting입니다. Permissions, Hooks, MCP, Commands를 모두 함께 사용하는 완성된 워크플로를 다룹니다.

Part 7

완료

Part 8

Integration (10p)

Part 9

Recap + Labs (10p)

~40 min

잔여 시간

## CHAPTER 04 - PART 08

# Settings – 트러블슈팅

settings.json, Hooks, MCP 심화





# Part 8 / 통합 패턴 & Troubleshooting

## 4가지 메커니즘의 완성된 결합

### PART OBJECTIVE

Permissions, Hooks, MCP, Commands 4가지 메커니즘을 모두 활용하는 통합 워크플로 패턴과 일반적인 문제 진단 방법을 다룹니다.

#### TOPIC 1

##### 통합 워크플로

4 메커니즘을 함께 사용한  
완성된 자동화 시나리오

#### TOPIC 2

##### 성능 최적화

Hook 오버헤드, MCP 응답  
시간, 컨텍스트 크기 관리

#### TOPIC 3

##### 보안 체크

8가지 보안 점검 항목과 권  
장 조치

#### TOPIC 4

##### Troubleshooting

일반적 문제 6가지와 단계  
별 진단/해결

10 Slides

Part 8 분량

~20 min

예상 강의 시간

4 mechanisms

통합

Production

운영 품질

# 4 메커니즘 통합 예시

## 완성된 PR 워크플로

# 시나리오: PR 자동 종합 처리

```
# 1. Custom Command (.claude/commands/handle-pr.md)
---
description: PR을 종합 처리 (리뷰 + 머지 + 알림 + JIRA)
allowed-tools:
  - Bash(gh pr:*)
  - mcp__jira__transition_issue
  - mcp__slack__send_message
---
```

PR #\$ARGUMENTS 를 종합 처리합니다:  
1. /review-pr 수행 (위 명령 활용)  
2. 승인 시 PR 머지  
3. JIRA 티켓 Done 처리 (MCP github + jira)  
4. Slack 알림 (MCP slack)

```
# 2. Permissions (managed-settings.json)
{
  "permissions": {
    "deny": ["Bash(rm -rf:*)", "Read(**/.env*)"],
    "allow": ["Bash(gh pr:*)", "Bash(git diff:*)"]
  }
}
```

# 3. Hooks (조직 표준 적용)

```
{
  "hooks": {
    "PreToolUse": [{ "matcher": ".*", "hooks": [
      { "command": "/opt/claude/dlp.sh" },
      { "command": "/opt/claude/audit.sh pre" }
    ]}],
    "PostToolUse": [{ "matcher": "Write|Edit", "hooks": [
      { "command": "/opt/claude/auto-format.sh" }
    ]}]
  }
}
```

# 4. MCP (사내 + 공식)

```
{
  "mcpServers": {
    "github": { "command": "npx", "args": ["@modelcontextprotocol/server-github"] },
    "jira": { "command": "npx", "args": ["@modelcontextprotocol/server-jira"] },
    "slack": { "command": "npx", "args": ["@modelcontextprotocol/server-slack"] }
  }
}
```

integration

Command

진입점

Permissions

안전망

Hooks

자동화

MCP

외부 통합

# 성능 최적화 종합

## 4 메커니즘별 튜닝

| 메커니즘         | 최적화 포인트                         | 효과          |
|--------------|---------------------------------|-------------|
| Permissions  | 구체적 패턴 우선, 와일드카드 회피             | 매칭 속도       |
| Hooks        | 동기/비동기 분리, jq 한 번, early return | 도구 호출 지연 감소 |
| MCP stdio    | 필요한 서버만 등록, 결과 캐싱               | 시작 시간 단축    |
| MCP http/sse | connection pooling, timeout 설정  | 네트워크 효율     |
| Commands     | allowed-tools 명시, model 최적 선택   | 의도된 동작 보장   |

### MEASURE FIRST

OpenTelemetry로 각 메커니즘의 응답 시간을 측정한 후 가장 느린 부분부터 최적화하세요. 추측이 아닌 데이터로 결정합니다.

### Specific

Perm 패턴

### Background

Hook 비동기

### Cache

MCP 재사용

### Measure

측정 우선

# 보안 체크리스트

## 8가지 점검 항목

| No | 항목                              | 확인                  |
|----|---------------------------------|---------------------|
| 1  | managed-settings deny에 위험 명령 포함 | rm, curl, sudo 등    |
| 2  | 환경 파일 읽기 차단                     | Read(**/.env*) deny |
| 3  | PreToolUse DLP Hook 적용          | 민감 정보 차단            |
| 4  | Audit logging Hook 활성화          | 모든 활동 SIEM          |
| 5  | MCP allowlist로 서버 통제            | mcpPolicy 활용        |
| 6  | MCP 토큰 환경변수 사용                  | 평문 저장 X             |
| 7  | 사내 npm 미러 강제                    | managed-settings    |
| 8  | Custom Command allowed-tools 명시 | 권한 최소화              |

8 items  
체크리스트

Layered  
계층 방어

Mandatory  
필수 사항

Quarterly  
주기 검토

# Troubleshooting / Hooks

## Hook이 실행되지 않을 때

hooks debug

```
# 증상: PostToolUse Hook이 실행되지 않음
# 진단 절차
# 1. settings 로딩 확인
$ claude doctor
# Hooks 섹션에 등록된 hook이 표시되는가?

# 2. matcher 패턴 검증
> /permissions # 이 명령은 권한 확인이지만
# 등록된 hooks는 doctor 출력으로 확인

# 3. 수동 실행 테스트
$ echo '{
  "session_id": "test",
  "tool_name": "Write",
  "tool_input": {
    "file_path": "/tmp/test.js",
    "content": "..."
  }
}' | /opt/claude/hooks/auto-lint.sh

# exit code 확인
$ echo $?
0 # 정상 (0이어야 함)
```

```
# 4. --debug 모드로 실시간 추적
$ claude --debug
> [Write 도구 사용]
# stderr에 다음이 표시되어야:
# [hook] PostToolUse matched
# [hook] Running /opt/claude/hooks/auto-lint.sh
# [hook] Exit code: 0

# 5. 일반 원인
- matcher가 너무 좁음 (Write 대신 Edit 호출 등)
- 스크립트 실행 권한 없음 (chmod +x)
- 스크립트 절대 경로가 아님
- timeout이 너무 짧음
- jq 같은 의존성 미설치
```

doctor

1차 진단

Manual

수동 테스트

--debug

실시간 추적

chmod +x

실행 권한

# Troubleshooting / MCP

## MCP 서버 연결 실패

mcp debug

# 증상: /mcp 명령에서 jira 서버가 'Failed'

# 진단 절차

# 1. /mcp로 상태 확인

```
> /mcp
jira [http] x Failed
Error: 401 Unauthorized
Last attempt: 2 minutes ago
```

# 2. 토큰 확인

```
$ echo $JIRA_TOKEN # 환경변수 설정 여부
$ # 토큰 만료 여부 (API 직접 호출로 확인)
$ curl -H "Authorization: Bearer $JIRA_TOKEN" \
  "https://jira.atlassian.com/api/myself"
# 401이면 토큰 만료, 200이면 정상
```

# 3. 로그 확인

```
> /mcp logs jira
# 최근 stderr 출력
```

# 4. 재연결

```
> /mcp reconnect jira
```

# 5. settings.json 검증

```
$ jq . .claude/settings.json
# JSON 문법 오류 확인
```

# 6. 일반 원인

- 환경변수 미설정 또는 만료
- URL 오타
- 네트워크/방화벽 차단
- 서버 측 문제 (curl로 직접 확인)
- npm 패키지 버전 호환 안 됨

# 7. stdio 서버는 프로세스 확인

```
$ ps aux | grep mcp
# 자식 프로세스가 살아있는가?
```

/mcp  
상태

curl  
직접 검증

logs  
로그

reconnect  
재연결

# Troubleshooting / Permission

## 권한 거부 오류

# 증상: Edit 도구가 권한 거부됨

# 1. /permissions로 확인

```
> /permissions Edit(src/api/user.js)
→ DENY (matched by 'Edit(src/api/secure/**)'
      from managed-settings.json)
```

# 2. 어느 계층 deny인지 확인

managed-settings.json: 조직 강제 → root만 수정 가능  
.claude/settings.json: 팀 공유 → PR로 수정  
~/.claude/settings.json: 개인 → 직접 수정 가능

# 3. 의도된 차단이라면

# 권한이 있는 사람에게 요청하거나  
# 다른 디렉토리에서 작업

# 4. 의도되지 않은 차단이라면 수정

# 예시: 너무 광범위한 deny 패턴  
"deny": ["Edit(src/\*\*/secure\*)"] # 의도

# 실제로는 src/api/user.js에 매칭 안 되어야 하는데  
# 패턴 오타나 잘못된 그룹으로 매칭됨

# 5. 수정 후 검증

```
> /permissions Edit(src/api/user.js)
→ ALLOW (matched by 'Edit(src/**)' from project settings)
```

# 6. 다른 일반 원인

- additionalDirectories에 추가 필요
- 절대 경로 매칭 문제
- managed-settings의 deny가 너무 광범위

perm debug

/permissions

진단 명령

Layer

계층 확인

Intent

의도 검증

Pattern

패턴 수정

# Troubleshooting / Performance

## 느린 응답 진단

### STEP 01

OTel  
메트릭 확인

### STEP 02

병목 영역  
식별

### STEP 03

메커니즘별  
튜닝

### STEP 04

재측정  
검증

### STEP 05

문서화

| 병목         | 진단                     | 해결                 |
|------------|------------------------|--------------------|
| Hook 응답 느림 | /tmp/hook-perf.log     | 비동기 분리             |
| MCP 응답 느림  | OTel mcp_call_duration | 캐싱, 페이지네이션         |
| 토큰 폭증      | /cost 명령               | 컨텍스트 정리            |
| 컨텍스트 한계    | doctor의 context_usage  | /clear 또는 Subagent |

Measure

측정

Identify

식별

Tune

튜닝

Verify

검증



# 일반 트러블슈팅 흐름

## 문제 발생 시 진단 순서

STEP 01

claude doctor

STEP 02

관련 명령  
(/permissions, /mcp)

STEP 03

--debug 모드

STEP 04

로그 확인

STEP 05

에스컬레이션

LEVEL 1

자가 진단

doctor, /permissions,  
/mcp, --debug로 대부분  
해결

LEVEL 2

팀 슬랙

동료에게 질문, 사내 위키 검  
색, 비슷한 사례 찾기

LEVEL 3

사내 IT

네트워크, 프록시, 인증서 같은  
사내 환경 문제

LEVEL 4

Anthropic Support

Claude Code 자체 문제,  
공식 지원 채널

Self

L1

Team

L2

IT

L3

Vendor

L4

# Part 8 Recap

## 통합과 Troubleshooting 핵심 정리

### TAKEAWAY 1

#### 4 메커니즘 통합

Permissions + Hooks +  
MCP + Commands를 함  
께 사용해 강력한 자동화

### TAKEAWAY 2

#### 성능 최적화

측정 우선, 메커니즘별 튜닝,  
OTel로 지속 모니터링

### TAKEAWAY 3

#### 보안 8 체크

deny, DLP, audit,  
allowlist, env, mirror,  
allowed-tools

### TAKEAWAY 4

#### 진단 흐름

doctor → /permissions  
→ /mcp → --debug → 에  
스컬레이션

### NEXT - PART 9

Part 9은 Chapter 4 핵심 요약과 4가지 Hands-on Lab입니다. 실제로 손에 익히는 마지막 시간입니다.

Part 8

완료

Part 9

Recap + 4 Labs

4 Labs

실습

~40 min

잔여

CHAPTER 04 – PART 09

# Settings – Summary & LAB

settings.json, Hooks, MCP 심화



# Chapter 4 핵심 요약

## 9개 Part 130장의 압축본

### SETTINGS

settings +  
permissions

3단 계층 이해,  
allow/ask/deny 패턴, 글  
롭과 명령 패턴 마스터

### HOOKS

4가지 시점 + 5 패턴

Pre/PostToolUse/Sessio  
nStart/Stop +  
DLP/Audit/Lint/Format/  
Notify

### MCP

5 공식 + 사내 구축

filesystem/github/slack/  
postgres/jira +  
TypeScript SDK 직접 작성

### COMMANDS

슬래시 명령 자산화

.md + frontmatter +  
\$ARGUMENTS, 팀 라이브  
러리 구축

### OUTCOME

여러분은 이제 Claude Code를 본인 워크플로에 깊이 커스터마이징하고 팀 표준 자동화 자산을 구축할 수 있습니다.

9개 Part의 모든 패턴이 즉시 적용 가능합니다.

9 Parts

완료

140 Slides

Chapter 4

~4h

강의 시간

4 Labs

실습 시작

# Hands-on Labs / 4종

## 직접 손에 익히는 시간

| Lab                          | 내용                                | 예상 시간 |
|------------------------------|-----------------------------------|-------|
| Lab 1 / 첫 settings.json      | permissions, model, theme 작성 + 검증 | 20분   |
| Lab 2 / PreToolUse Hook      | DLP 스크립트 작성 + Hook 등록 + 차단 검증     | 25분   |
| Lab 3 / 사내 MCP 서버            | TypeScript로 간단한 사내 MCP 서버 작성      | 30분   |
| Lab 4 / Custom Slash Command | 팀 명령 1개 작성 + Git 공유 + 데모          | 25분   |

### TOTAL

4개 랩 합산 약 100분. 본인 실 작업 환경에서 진행하면 실무 적용도가 높아집니다. 어려움이 있을 때 즉시 AWS에게 도움 요청 가능.

4 Labs

실습 수

~100 min

총 시간

Hands-on

실습 중심

Mentor

도움 가능

# Lab 1 / 첫 settings.json

권한 + 모델 + 테마 (20분)

## STEP 01

디렉토리  
생성

## STEP 02

permissions  
작성

## STEP 03

model alias  
등록

## STEP 04

theme/editor  
선택

## STEP 05

doctor로  
검증

```
# 1. 사용자 전역 settings 디렉토리
$ mkdir -p ~/.claude
# 2. settings.json 작성
$ cat > ~/.claude/settings.json <<'EOF'
{
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob",
      "Bash(npm test:*)", "Bash(npm run lint)",
      "Bash(git diff:*)", "Bash(git log:*)", "Bash(git status)"
    ],
    "ask": ["Edit", "Write", "Bash(git push:*)"],
    "deny": [
      "Bash(rm -rf:*)", "Bash(curl:*)",
      "Read(**/.env*)", "Read(**/.ssh/**)"
    ]
  },
}
```

```
"model": "claude-sonnet-4-5",
"modelAlias": {
  "fast": "claude-haiku-4-5",
  "smart": "claude-opus-4-5"
},
"theme": "dark",
"outputStyle": "concise",
"editor": {
  "tabWidth": 2,
  "preferredQuotes": "single"
}
}
EOF
```

```
# 3. 검증
$ claude doctor
# Settings 섹션 확인 - 모든 설정이 인식되어야 함

# 4. 실제 사용 테스트
$ claude
> /permissions
> /model fast # 모델 전환 테스트
```

Lab 1

20 min  
예상 소요

User-level  
사용자 전역

Verify  
검증

First steps  
입문

# Lab 2 / PreToolUse Hook

## DLP 스크립트 구현 (25분)

Lab 2

```
# 1. Hook 스크립트 작성
$ mkdir -p ~/.claude/hooks
$ cat > ~/.claude/hooks/my-dlp.sh <<'EOF'
#!/bin/bash
INPUT=$(cat)
TOOL=$(echo "$INPUT" | jq -r '.tool_name')
# Bash 명령 검사
if [ "$TOOL" = "Bash" ]; then
  CMD=$(echo "$INPUT" | jq -r '.tool_input.command')
  if echo "$CMD" | grep -qE 'rm -rf|chmod 777'; then
    echo '{"action":"block","reason":"위험 명령 감지"}' >&2
    exit 1
  fi
fi

# Edit/Write 콘텐츠 검사
if [ "$TOOL" = "Write" ] || [ "$TOOL" = "Edit" ]; then
  CONTENT=$(echo "$INPUT" | jq -r \
    '.tool_input.content // .tool_input.new_string')
  if echo "$CONTENT" | grep -qE 'AKIA[0-9A-Z]{16}'; then
    echo '{"action":"block","reason":"AWS Key 감지"}' >&2
    exit 1
  fi
fi
exit 0
EOF
```

```
$ chmod +x ~/.claude/hooks/my-dlp.sh
```

```
# 2. settings.json에 Hook 등록
$ cat >> ~/.claude/settings.json # 또는 직접 편집
# hooks 필드 추가: PreToolUse matcher "Bash|Edit|Write"
# command: "~/.claude/hooks/my-dlp.sh"
```

```
# 3. 차단 검증
$ claude
> 다음을 src/aws.js에 작성해줘:
  const key = "AKIAIOSFODNN7EXAMPLE";
# 결과: Hook이 차단해야 정상
```

```
# 4. 통과 검증
> README.md에 "Hello World"를 추가해줘
# 결과: 정상 통과
```

25 min  
예상 소요

2 patterns  
rm + AWS Key

Block test  
차단 검증

Pass test  
통과 검증

# Lab 3 / 사내 MCP 서버

## TypeScript로 작성 (30분)

### Lab 3

```
# 1. 프로젝트 생성
$ mkdir my-mcp && cd my-mcp
$ npm init -y
$ npm install @modelcontextprotocol/sdk
$ npm install -D typescript @types/node tsx
# 2. src/index.ts 작성 (간단한 사내 도구)
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { ListToolsRequestSchema, CallToolRequestSchema }
  from "@modelcontextprotocol/sdk/types.js";

const server = new Server(
  { name: "my-mcp", version: "1.0.0" },
  { capabilities: { tools: {} } }
);

server.setRequestHandler(ListToolsRequestSchema, async () => ({
  tools: [{
    name: "company_random_fact",
    description: "사내 trivia 무작위 반환",
    inputSchema: { type: "object", properties: {} },
  }],
}));
```

```
server.setRequestHandler(CallToolRequestSchema, async (req) => {
  const facts = [
    "우리 회사는 2010년에 설립되었습니다.",
    "코드 리뷰는 24시간 내에 첫 응답을 줍니다.",
    "Friday는 No Meeting Day입니다.",
  ];
  return { content: [{
    type: "text",
    text: facts[Math.floor(Math.random() * facts.length)]
  }]}
});

const transport = new StdioServerTransport();
await server.connect(transport);

# 3. settings.json에 등록 후 사용
> 사내 trivia 하나 알려줘
```

30 min

예상 소요

Simple

단순 예제

End-to-end

전 과정

Working

실제 동작



# Lab 4 / Custom Slash Command

## 팀 명령 작성 (25분)

Lab 4

```
# 1. 프로젝트에 .claude/commands 디렉토리 생성
$ cd ~/my-project
$ mkdir -p .claude/commands
```

```
# 2. 명령 파일 작성 (.claude/commands/standup.md)
$ cat > .claude/commands/standup.md <<'EOF'
```

```
---
description: 매일 stand-up 자동 생성 (어제 한 일 + 오늘 할 일)
allowed-tools:
  - Bash(git log:*)
  - Read
---
```

# 1. 어제 한 일 - 본인의 어제 commit 추출  
어제 (yesterday) 본인의 git commit 목록을 가져와서  
사람이 읽기 쉬운 형태로 정리해 주세요.

# 2. 오늘 할 일 - \$ARGUMENTS 분석  
\$ARGUMENTS 가 있다면 오늘 할 일 후보입니다.  
없다면 진행 중인 PR이나 issue를 살펴보고 추천해 주세요.

```
# 3. 출력 형식
## Standup - 2026-05-25
```

```
### 어제
- ✅ <commit message 1>
- ✅ <commit message 2>
```

```
### 오늘
- 🍌 <task 1>
- 🍌 <task 2>
```

```
### 블로커
- (있다면) <blocker description>
```

복사해서 Slack에 붙여넣기 좋은 형식으로 작성.  
EOF

```
# 3. 사용
> /standup
> /standup "이슈 #123 마무리, 코드 리뷰 2건"
```

```
# 4. Git에 공유
$ git add .claude/commands/standup.md
$ git commit -m "feat: add /standup command"
$ git push
```

```
# 5. 팀 데모 (선택)
```

25 min

예상 소요

Practical

실용 명령

Git share

팀 공유

Demo

발표

# 자주 묻는 질문

## Chapter 4 관련 FAQ

### FAQ

Q1. settings.json을 잘못 설정해서 망가지면?

A. 파일을 삭제하면 기본값으로 복원됩니다.  
rm ~/.claude/settings.json 후 claude doctor로 확인.  
managed-settings는 관리자에게 요청.

Q2. Hooks가 너무 느려서 작업이 답답한데?

A. /tmp/hook-perf.log에 실행 시간을 기록해 측정.  
무거운 작업은 백그라운드(&)로 비동기 분리.  
timeout 조정으로 임시 완화 가능.

Q3. MCP 서버를 npm 미러에 두려면?

A. npm pack으로 tarball 생성 후 사내 미러에 publish.  
settings.json에서 사내 registry로 자동 사용됨.

Q4. 같은 이름의 명령이 여러 곳에 있으면?

A. 우선순위: 프로젝트 > 사용자 > 빌트인.  
더 상위가 적용되며 /commands로 확인 가능.

Q5. Subagent와 Slash Command 중 선택은?

A. 격리된 큰 분석은 Subagent.  
빠른 워크플로 자동화는 Command.  
한 Command 안에서 Subagent 호출도 가능.

Q6. Hook을 한 환경에만 적용하려면?

A. Hook 스크립트 안에서 환경변수나 hostname으로 분기.  
또는 .claude/settings.local.json에 등록 (Git 미커밋).

### 6 Q

대표 질문

### Docs

공식 문서

### Slack

팀 채널

### Practice

실무

# 워크샵 후 실행 항목

오늘 배운 것을 내일 실무에

## STEP 01

오늘  
첫 settings

## STEP 02

1주차  
DLP Hook

## STEP 03

2주차  
MCP 1개

## STEP 04

3주차  
Command 5개

## STEP 05

월말  
팀 공유

## WEEK 1

### 개인 환경

~/.claude/settings.json  
작성, Lab 1 패턴 적용, 일  
주일 사용

## WEEK 2

### 자동화

PostToolUse auto-  
format, PreToolUse DLP  
작성과 적용

## WEEK 3

### MCP 통합

회사에서 자주 쓰는 도구 1  
개를 MCP로 통합 (사내  
wiki, 사내 API 등)

## WEEK 4

### 팀 자산

팀 표준 명령 3-5개를  
.claude/commands로 작  
성, Git 공유, 데모

Day 1  
Settings

Week 1  
Hook

Week 2  
MCP

Week 3  
Command

# References

## 참고 자료 및 출처

### REFERENCES

#### Official Documentation

<https://code.claude.com/docs/ko/settings>  
<https://code.claude.com/docs/ko/hooks>  
<https://code.claude.com/docs/ko/mcp>  
<https://code.claude.com/docs/ko/slash-commands>

#### MCP Resources

<https://modelcontextprotocol.io/>  
<https://github.com/modelcontextprotocol/servers>  
<https://github.com/punkpeye/awesome-mcp-servers>  
<https://mcp.so/>

#### TypeScript MCP SDK

<https://github.com/modelcontextprotocol/typescript-sdk>  
<https://www.npmjs.com/package/@modelcontextprotocol/sdk>

#### Hook 패턴 참고

<https://github.com/anthropic/claude-code-hooks-examples>

#### 내부 자산 (사내)

<https://wiki.company.com/claude-code/>  
사내 Slack: #claude-code-help

#### 다음 챗터

Chapter 5 - CLI Reference (Day 3 AM)  
Chapter 6 - Agent SDK (Day 3 PM)

### Docs

공식 문서

### MCP

프로토콜

### SDK

구현 참고

### Community

예제 모음

# Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

[whchoi@amazon.com](mailto:whchoi@amazon.com)

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

